**ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH**
**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN**
**KHOA KỸ THUẬT MÁY TÍNH**

**Trịnh Nguyễn Thanh Bình - 23520162**
**Bùi Quốc Bảo - 23520092**
**Diệp Thanh Phong - 23521160**
**Lê Trần Huỳnh Phong - 23521164**

**ĐỒ ÁN CHUYÊN NGÀNH**

# THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG PHÂN LOẠI MÀU SẮC TRÊN BĂNG TẢI SỬ DỤNG ĐIỀU KHIỂN PID CHO ĐỘNG CƠ DC

# DESIGN AND IMPLEMENTATION OF A COLOR SORTING SYSTEM ON A CONVEYOR BELT USING PID CONTROL FOR A DC MOTOR

TP. HỒ CHÍ MINH, NĂM 2025

**ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH**
**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN**
**KHOA KỸ THUẬT MÁY TÍNH**

**Trịnh Nguyễn Thanh Bình - 23520162**
**Bùi Quốc Bảo - 23520092**
**Diệp Thanh Phong - 23521160**
**Lê Trần Huỳnh Phong - 23521164**

**ĐỒ ÁN CHUYÊN NGÀNH**

# THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG PHÂN LOẠI MÀU SẮC TRÊN BĂNG TẢI SỬ DỤNG ĐIỀU KHIỂN PID CHO ĐỘNG CƠ DC

# DESIGN AND IMPLEMENTATION OF A COLOR SORTING SYSTEM ON A CONVEYOR BELT USING PID CONTROL FOR A DC MOTOR

**GIẢNG VIÊN HƯỚNG DẪN**
**Ths. Phạm Minh Quân**

TP. HỒ CHÍ MINH, NĂM 2025

# Lời cảm ơn

Với lòng biết ơn sâu sắc, nhóm em xin gửi lời cảm ơn chân thành nhất đến Thầy Phạm Minh Quân, Giảng viên hướng dẫn môn học Thiết kế Hệ thống nhúng, thuộc Khoa Kỹ thuật Máy tính, Trường Đại học Công nghệ thông tin.

Trong suốt quá trình thực hiện đồ án, Thầy đã tận tình truyền đạt kiến thức chuyên môn sâu rộng, định hướng nghiên cứu và tạo mọi điều kiện thuận lợi để nhóm em hoàn thành đề tài một cách tốt nhất. Sự hướng dẫn quý báu và những góp ý chi tiết từ Thầy không chỉ giúp em giải quyết các vấn đề kỹ thuật mà còn phát triển kỹ năng tư duy và làm việc độc lập.

Em cũng xin bày tỏ lòng tri ân sâu sắc đến Khoa Kỹ thuật Máy tính - Trường Đại học Công nghệ thông tin đã cung cấp môi trường học tập và nghiên cứu chất lượng, là nền tảng vững chắc cho việc thực hiện đồ án này.

Mặc dù đã nỗ lực hết mình, đồ án chắc chắn không tránh khỏi những thiếu sót. Nhóm em kính mong nhận được sự thông cảm và những ý kiến đóng góp quý báu từ Thầy để nghiên cứu được hoàn thiện hơn nữa.

Xin chân thành cảm ơn!

# Mục lục

**Tóm tắt đồ án** **1**

**1 Mở đầu** **2**

1.1 Lý do chọn đề tài . . . . . 2

1.2 Mục tiêu đề tài . . . . . 3

1.3 Đối tượng nghiên cứu . . . . . 3

1.4 Phạm vi nghiên cứu . . . . . 3

**2 Tổng quan đề tài** **4**

**3 Phương pháp nghiên cứu** **6**

3.1 Cơ sở lý thuyết . . . . . 6

3.1.1 Thiết kế Finite State Machine (FSM) . . . . . 6

3.1.1.1 Tổng quan về FSM . . . . . 6

3.1.1.2 Thiết kế FSM . . . . . 7

3.1.1.3 Các trạng thái của FSM . . . . . 10

3.1.2 Thiết kế Bộ điều khiển PID cho DC motor . . . . . 12

3.1.2.1 Tổng quan về DC motor . . . . . 12

3.1.2.2 Tổng quan về H-brigde . . . . . 13

3.1.2.3 PID . . . . . 15

3.1.3 Các phần cứng chuyên dụng . . . . . 19

3.1.3.1 Tổng quan về Color Sensor . . . . . 19
3.1.3.2 Tổng quan về Encoder . . . . . 21
3.1.3.3 Tổng quan về Servo . . . . . 23
3.1.3.4 Tổng quan về IR Infrared Obstacle Avoidance . . 24
3.1.3.5 Tổng quan về băng chuyền . . . . . 25
3.2 Phương pháp nghiên cứu . . . . . 26
3.2.1 Thiết kế sơ đồ hệ thống (System diagram) . . . . . 26
3.2.1.1 Các khối thành phần và vai trò . . . . . 27
3.2.1.2 Các mối quan hệ chính . . . . . 29
3.2.2 Sơ đồ khối của hệ thống (Block diagram) . . . . . 32
3.2.2.1 Vai trò và thành phần của từng tầng . . . . . 34
3.2.3 Sơ đồ tĩnh của hệ thống ( Static diagram ) . . . . . 37
3.2.3.1 Các module, API được sử dụng ở từng tầng. . . . 37
3.2.4 Thiết kế biểu đồ tuần tự (Sequence diagram) . . . . . 44
3.2.5 Thiết kế lưu đồ giải thuật . . . . . 48
3.2.5.1 Các bước logic, giai đoạn và quá trình vận hành . 49

**4 Kết quả và đánh giá 52**
4.1 Kết quả . . . . . 52
4.1.1 Sản phẩm băng chuyền . . . . . 52
4.1.2 Kết quả kiểm thử các chức năng . . . . . 53
4.1.2.1 Nhận diện màu sắc . . . . . 53
4.1.2.2 Bộ điều khiển băng tải PID . . . . . 54
4.1.2.3 Phân loại bằng servo . . . . . 55
4.1.2.4 Nội dung hiển thị LCD . . . . . 55
4.1.2.5 Nút điều khiển . . . . . 55
4.1.2.6 Bộ đếm sản phẩm (IR sensor) . . . . . 56
4.2 Đánh giá . . . . . 56

**5 Kết luận và hướng phát triển 58**

5.1 Hướng phát triển . . . . . 58
5.1.1 Phát triển về cơ khí . . . . . 58
5.1.2 Phát triển về bộ điều khiển tốc độ . . . . . 59
5.1.3 Phát triển về cảm biến . . . . . 59
5.2 Kết luận . . . . . 59

**TÀI LIỆU THAM KHẢO** **62**
Tài liệu tiếng Việt . . . . . 62
Tài liệu tiếng Anh . . . . . 63

# Danh sách hình vẽ


3.1 Thiết kế FSM theo kiến trúc Moore cho tầng trạng thái. . . . . 8
3.2 Thiết kế FSM theo kiến trúc Moore cho tầng chức năng. . . . . 9
3.3 Động cơ DC giảm tốc V1 Dual Shaft Plastic Geared TT Motor[4] . . . . 13
3.4 Mạch điều khiển động cơ L298 DC Motor Driver[6] . . . . . 14
3.5 Flow tính toán PID . . . . . 16
3.6 Cảm biến màu sắc $I^2C$ UART TCS34725 Color Sensor V1[1] . . . . . 20
3.7 Đĩa Encoder 20 xung [3] . . . . . 22
3.8 Mạch cảm biến tốc độ Encoder V1[7] . . . . . 22
3.9 Động cơ RC Servo 9G 0-180°[5] . . . . . 23
3.10 Cảm biến vật cản hồng ngoại IR Infrared Obstacle Avoidance[2] . . . . 24
3.11 Mô hình băng chuyền lần đầu. . . . . 25
3.12 Mô hình băng chuyền sau khi chỉnh sửa. . . . . 25
3.13 Tổng quan sơ đồ hệ thống. . . . . 27
3.14 Nguồn cấp cho cơ cấu chấp hành của hệ thống. . . . . 31
3.15 Tổng quan sơ đồ khối hệ thống. . . . . 33
3.16 Init Task . . . . . 45
3.17 Button Task . . . . . 45
3.18 Main Task . . . . . 46
3.19 Motor Task . . . . . 47
3.20 LCD Task . . . . . 47
3.21 Tổng quan lưu đồ giải thuật. . . . . 48

4.1 Hình ảnh đóng gói băng chuyền hoàn thiện . . . . . 53
4.2 Response Graph PID . . . . . 54

# Tóm tắt đồ án

Hệ thống băng tải phân loại màu sắc là một ứng dụng phổ biến, tuy nhiên vẫn còn tồn tại thách thức lớn trong việc đảm bảo tốc độ vận hành ổn định và chính xác.

Đây là yếu tố then chốt để định vị và phân loại vật thể thành công, đặc biệt khi hệ thống phải đối phó với nhiễu động từ môi trường. Để giải quyết vấn đề này, nhóm nghiên cứu đã xây dựng một hệ thống nhúng dựa trên vi xử lý ESP32, tích hợp cảm biến màu sắc, động cơ servo và cảm biến vật cản.

Giải pháp cốt lõi được áp dụng là thuật toán điều khiển PID (Proportional-Integral-Derivative), cho phép hệ thống tự động tính toán và điều chỉnh công suất động cơ để băng tải đạt được và duy trì tốc độ mong muốn với sai số tối thiểu, qua đó tối ưu hóa đáng kể hiệu suất và độ tin cậy của quá trình phân loại. Việc ứng dụng PID để điều chỉnh tốc độ băng chuyền đã giúp hệ thống phân loại đạt độ chính xác.

# Chương 1

# Mở đầu

## 1.1 Lý do chọn đề tài

Trong bối cảnh tự động hóa ngày càng phát triển, các hệ thống nhúng đóng vai trò quan trọng trong việc điều khiển và giám sát các quá trình công nghiệp. Đặc biệt, các hệ thống phân loại sản phẩm sử dụng băng chuyền được ứng dụng rộng rãi nhằm nâng cao năng suất và độ chính xác trong sản xuất.

Để hệ thống băng chuyền hoạt động ổn định và đáp ứng tốt yêu cầu phân loại màu sắc, việc thiết kế bộ điều khiển cho động cơ là yếu tố then chốt. Bộ điều khiển PID là phương pháp điều khiển phổ biến, dễ triển khai trên các nền tảng vi điều khiển và phù hợp với các hệ thống nhúng có tài nguyên hạn chế.

Xuất phát từ yêu cầu thực tiễn đó, nhóm chúng em lựa chọn đề tài: “Thiết kế bộ điều khiển PID cho hệ thống phân loại màu sắc sử dụng băng chuyền”. Đề tài nhằm nghiên cứu, thiết kế và triển khai bộ điều khiển PID trên hệ thống để nâng cao tính ổn định và độ chính xác của quá trình phân loại.

## 1.2 Mục tiêu đề tài

Mục đích của đề tài là nghiên cứu và thiết kế hệ thống phân loại màu sắc sử dụng băng chuyền, trong đó bộ điều khiển PID được áp dụng để điều khiển tốc độ động cơ nhằm đảm bảo hệ thống hoạt động ổn định và chính xác. Thông qua đề tài, nhóm hướng tới việc vận dụng kiến thức đã học về thiết kế hệ thống cũng như điện – điện tử và cảm biến vào mô hình thực tế, đồng thời đánh giá hiệu quả của bộ điều khiển PID trong quá trình phân loại màu.

## 1.3 Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài bao gồm:

- Hệ thống phân loại màu sắc sử dụng băng chuyền với động cơ DC.
- Bộ điều khiển PID giúp điều khiển tốc độ băng chuyền.
- Cảm biến màu sắc, servo, động cơ DC và quá trình thu nhập, xử lý tín hiệu phục vụ cho quá trình phân loại.

## 1.4 Phạm vi nghiên cứu

Đề tài được thực hiện trong phạm vi đồ án cuối kỳ của một môn học, tập trung vào việc thiết kế và xây dựng mô hình hệ thống phân loại màu sắc sử dụng băng chuyền ở quy mô nhỏ. Nội dung nghiên cứu bao gồm xây dựng cấu trúc hệ thống, thiết kế và hiệu chỉnh bộ điều khiển PID, cũng như đánh giá hoạt động của hệ thống thông qua các thí nghiệm cơ bản. Các nội dung mở rộng và tối ưu hóa nâng cao không thuộc phạm vi của đề tài.

# Chương 2

# Tổng quan đề tài

Băng tải là một hệ thống cơ khí thiết yếu, được ứng dụng rộng rãi trong các quy trình sản xuất và hoạt động xếp dỡ hàng hóa nhằm mục đích vận chuyển vật liệu [9]. Trong các hệ thống phân loại, việc kiểm soát tốc độ băng tải là yếu tố then chốt. Tốc độ ổn định là điều kiện tiên quyết để các cảm biến có thể thu thập và xử lý dữ liệu chính xác, giảm thiểu nhiễu động, đồng thời đảm bảo sự ổn định chung của dây chuyền sản xuất, cho phép công nhân kịp thời khắc phục sự cố.

Các hệ thống này thường được dẫn động bằng động cơ một chiều (DC). Để điều chỉnh tốc độ, bộ phận điều khiển sẽ liên tục tính toán độ lệch giữa tốc độ mong muốn và tốc độ thực tế. Tín hiệu điều khiển sau đó được tạo ra thông qua nhiều phương pháp khác nhau, bao gồm các thuật toán thông minh, Bộ điều khiển Logic Khả trình (PLC - Programmable Logic Controller) – đóng vai trò "bộ não"điều khiển các quá trình cơ khí, Bộ điều khiển PID, hoặc các thuật toán kết hợp.

Đã có nhiều nghiên cứu áp dụng các giải pháp này, chẳng hạn như Umoren và cộng sự [10] đã ứng dụng bộ điều khiển PID cho băng tải công nghiệp nhằm đồng bộ hóa tốc độ giữa các công đoạn trong nhà máy bia. Tương tự, Ufot và cộng sự [9] đã sử dụng Bộ điều khiển Logic Mờ (FLC) để điều khiển đồng bộ tốc độ dây chuyền. Tuy

nhiên, các hệ thống hiện tại vẫn cần được cải thiện hiệu năng và giảm sai số truyền, cho phép vật liệu di chuyển nhanh hơn và đồng bộ hơn với tốc độ vận hành của toàn bộ dây chuyền.

Do đó, nhóm nghiên cứu tập trung vào việc thiết kế bộ điều khiển PID cho động cơ DC và triển khai trên hệ thống phân loại màu sắc. Mục tiêu là tối ưu hóa hiệu quả điều khiển, đảm bảo hệ thống đạt độ chính xác cao nhất trong quá trình phân loại.

# Chương 3

# Phương pháp nghiên cứu

## 3.1 Cơ sở lý thuyết

Hệ thống phân loại màu sắc hoạt động chủ yếu dựa trên hai nguyên tắc điều khiển cốt lõi: Mô hình Máy trạng thái hữu hạn (Finite State Machine - FSM ) để quản lý các trạng thái của hệ thống cũng như luồng logic, và Bộ điều khiển PID giúp tối ưu hóa tốc độ động cơ DC. Ngoài ra, hệ thống còn tích hợp các phần cứng chuyên dụng như (cảm biến, động cơ DC, servo và băng tải). Sự phối hợp nhằm đảm bảo sự ổn định toàn hệ thống và đạt hiệu suất hoạt động cao và chính xác nhất.

### 3.1.1 Thiết kế Finite State Machine (FSM)

#### 3.1.1.1 Tổng quan về FSM

Máy trạng thái hữu hạn (Finite State Machine – FSM) là một mô hình trừu tượng dùng để biểu diễn hoạt động của một hệ thống chỉ có thể tồn tại trong một số hữu hạn các trạng thái. Ở mỗi trạng thái, hệ thống tạo ra một hoặc nhiều tín hiệu đầu ra, và khi xảy ra một sự kiện nhất định, nó sẽ chuyển từ trạng thái này sang trạng thái khác. Việc lựa chọn trạng thái kế tiếp có thể phụ thuộc vào các tín hiệu đầu vào. Xét về bản chất, FSM thể hiện chức năng tương tự như cách một chương trình hay một máy tính

vận hành và xử lý thông tin.

FSM có hai kiến trúc phổ biến là Moore và Mealy. Ở kiến trúc Moore, các tín hiệu đầu ra chỉ phụ thuộc vào trạng thái hiện tại của hệ thống, còn trạng thái kế tiếp được xác định dựa trên trạng thái hiện tại kết hợp với các tín hiệu đầu vào. Trong khi đó, ở kiến trúc Mealy, các tín hiệu đầu ra phụ thuộc đồng thời vào trạng thái hiện tại và các tín hiệu đầu vào, và trạng thái kế tiếp cũng là hàm của trạng thái hiện tại và đầu vào. Sự khác biệt này tạo nên đặc điểm riêng về cách hoạt động và thiết kế của mỗi loại FSM[8].

#### 3.1.1.2 Thiết kế FSM

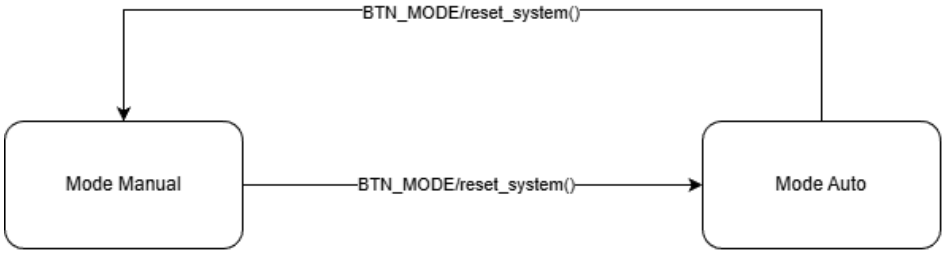
Do hệ thống của nhóm có hai chế độ hoạt động và mỗi chế độ hoạt động có những tín hiệu khác nhau vì thế nhóm đã thiết kế hệ thống theo kiến trúc phân cấp ( Hierarchical FSM ), trong đó FSM được chia thành hai tầng chức năng khác nhau quản lý mức độ điều khiển của hệ thống. Ở tầng trên, Mode FSM quản lý chế độ vận hành của hệ thống (MANUAL và AUTO). Ở tầng dưới, Operation FSM điều khiển chu trình hoạt động của băng chuyền, bao gồm các trạng thái IDLE, CLASSIFYING và CONVEYOR_RUNNING.

Cả hai tầng FSM đều được thiết kế theo kiến trúc Moore, nghĩa là các hành động và đầu ra của hệ thống được xác định bởi trạng thái hiện tại, trong khi các tín hiệu đầu vào chỉ đóng vai trò kích hoạt việc chuyển trạng thái.

**1.Mode FSM**

Ở tầng trạng thái chỉ có hai trạng thái quản lý chế độ của hệ thống. Hệ thống mặc định bắt đầu ở trạng thái Manual và khi nhận được tín hiệu từ Button Mode thì hệ thống sẽ thực hiện $reset_system()$.

**Lưu ý:** FSM của hệ thống được thiết kế theo kiến trúc Moore, trong đó các đầu ra chính của hệ thống được xác định bởi trạng thái hiện tại. Các hành động gắn với mũi tên chuyển trạng thái trong sơ đồ UML chỉ mang tính quản lý trạng thái và không ảnh hưởng đến bản chất Moore của FSM.




Hình 3.1: Thiết kế FSM theo kiến trúc Moore cho tầng trạng thái.

### 2.Operation FSM

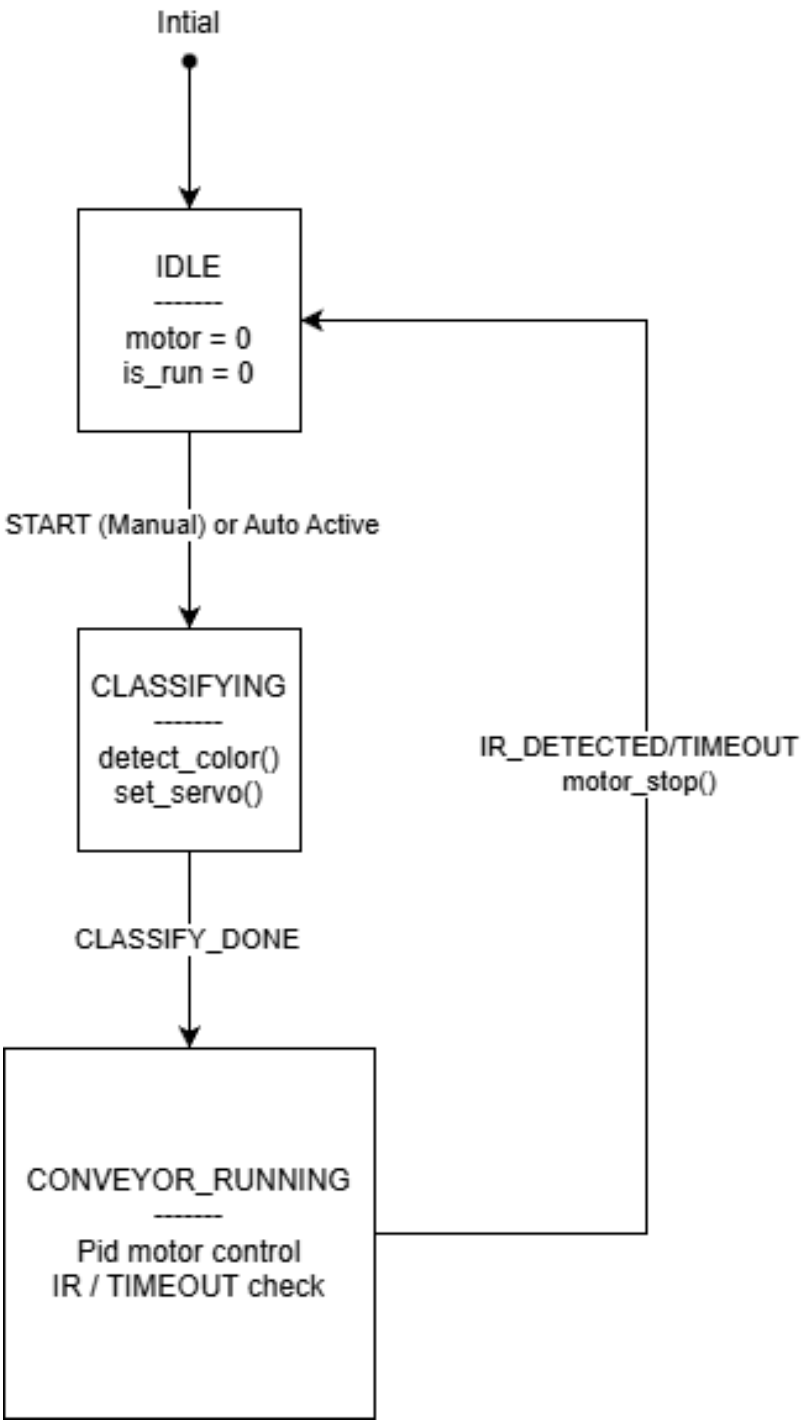
### Bảng trạng thái:

| Current State | Mode | Event | Guard | Next State |
|---|---|---|---|---|
| S0 | Manual | BTN_START | g_start_request == true | S1 |
| S0 | Auto | AUTO_ACTIVE | – | S1 |
| S1 | All | CLASSIFY_DONE | – | S2 |
| S2 | All | IR_DETECTED | – | S0 |
| S2 | Manual | TIMEOUT | – | S0 |

### Encoding

| State | Action |
|---|---|
| S0 | IDLE |
| S1 | CLASSIFYING |
| S2 | COVEYOR_RUNNING |

Bảng 3.1: Bảng Encoding




Hình 3.2: Thiết kế FSM theo kiến trúc Moore cho tầng chức năng.

#### 3.1.1.3 Các trạng thái của FSM

FSM của hệ thống được phân cấp thành hai tầng chính: Mode FSM và Operation FSM. Mode FSM chịu trách nhiệm quản lý chế độ vận hành của hệ thống, trong khi Operation FSM điều khiển chu trình hoạt động của băng chuyền.

**1.Các trạng thái của Mode FSM**

Mode FSM bao gồm hai trạng thái là MODE_MANUAL và MODE_AUTO, quyết định cách thức kích hoạt và vận hành chu trình phân loại.

Trong **MODE_MANUAL**, hệ thống hoạt động theo từng chu kỳ và chỉ được kích hoạt khi có yêu cầu hợp lệ từ người dùng. Operation FSM khởi đầu tại trạng thái IDLE, nơi băng tải dừng và hệ thống chờ tín hiệu khởi động. Khi xảy ra sự kiện BTN_START và điều kiện bảo vệ $g_start_request$ == true được thỏa mãn, FSM chuyển sang trạng thái CLASSIFYING để thực hiện nhận diện màu. Sau khi quá trình nhận diện hoàn tất, sự kiện CLASSIFY_DONE kích hoạt chuyển sang trạng thái CONVEYOR_RUNNING, trong đó băng tải chạy đúng một chu kỳ. Chu kỳ này kết thúc khi cảm biến hồng ngoại phát hiện sản phẩm (IR_DETECTED) hoặc khi xảy ra TIMEOUT, sau đó FSM quay trở về trạng thái IDLE để chờ lệnh tiếp theo.

Trong MODE_AUTO, hệ thống vận hành liên tục và không phụ thuộc vào nút khởi động từ người dùng. Operation FSM không duy trì trạng thái IDLE trong thời gian dài mà tự động kích hoạt chu trình phân loại. Trạng thái CLASSIFYING chỉ diễn ra trong thời gian ngắn để xác định màu của vật thể mới khi cơ cấu phân loại đang ở trạng thái mặc định. Sau đó, FSM chuyển sang trạng thái CONVEYO_RUNNING với tốc độ cố định. Mỗi khi xảy ra sự kiện IR_DETECTED, hệ thống cập nhật bộ đếm sản phẩm và nhanh chóng quay lại vòng lặp phân loại tiếp theo mà không cần dừng hoàn toàn băng tải.

**2.Các trạng thái của Operation FSM**

**Trạng thái IDLE** là trạng thái chờ của hệ thống, trong đó động cơ băng tải dừng và hệ thống không thực hiện phân loại. Trạng thái này đóng vai trò điểm bắt đầu và kết thúc của mỗi chu kỳ trong MODE_MANUAL, đồng thời đảm bảo hệ thống luôn ở trạng thái an toàn trước khi chuyển sang các trạng thái hoạt động khác.

**Trạng thái CLASSIFYING** là giai đoạn hệ thống thực hiện nhận diện màu sắc của sản phẩm. Trong trạng thái này, dữ liệu từ cảm biến màu được thu thập và xử lý để xác định màu chủ đạo, sau đó các servo được điều khiển nhằm thiết lập hướng phân loại phù hợp. Trạng thái CLASSIFYING kết thúc khi quá trình nhận diện hoàn tất và phát sinh sự kiện CLASSIFY_DONE.

**Trạng thái CONVEYOR_RUNNING** là giai đoạn băng tải hoạt động để vận chuyển sản phẩm. Động cơ được điều khiển bằng bộ PID nhằm duy trì tốc độ ổn định, đồng thời hệ thống giám sát các cảm biến hồng ngoại để phát hiện sản phẩm hoặc xác định điều kiện TIMEOUT. Khi một trong các sự kiện này xảy ra, FSM kết thúc chu kỳ vận chuyển và chuyển về trạng thái IDLE trong MODE_MANUAL, hoặc tiếp tục vòng lặp phân loại trong MODE_AUTO.
Các tín hiệu đầu vào và sự kiện FSM:

- BTN_START: Nút nhấn khởi động chu trình phân loại trong MODE_MANUAL. Sự kiện này chỉ có hiệu lực khi điều kiện bảo vệ g_start_request == true được thỏa mãn, dùng để kích hoạt chuyển từ trạng thái IDLE sang CLASSIFYING. Tại MODE_AUTO, nút nhấn này mang nhiệm vụ thay đổi tốc độ setpoint (được tính bằng *RPS*) của motor.
- BTN_MODE: Nút nhấn này thay đổi chế độ giữa MODE_AUTO và MODE_MANUAL của hệ thống.
- CLASSIFY_DONE: Sự kiện nội bộ phát sinh khi hệ thống hoàn tất quá trình nhận diện màu và thiết lập cơ cấu phân loại, dùng để kích hoạt chuyển từ trạng thái CLASSIFYING sang CONVEYOR_RUNNING.
- IR_DETECTED: Sự kiện xảy ra khi cảm biến hồng ngoại phát hiện sản phẩm đi qua. Sự kiện này dùng để kết thúc chu kỳ vận chuyển và kích hoạt chuyển trạng thái trong Operation FSM.

- TIMEOUT: Sự kiện bảo vệ phát sinh khi băng tải hoạt động vượt quá thời gian cho phép mà không phát hiện sản phẩm. Sự kiện này đảm bảo an toàn cho hệ thống và chỉ áp dụng trong MODE_MANUAL.

### 3.1.2 Thiết kế Bộ điều khiển PID cho DC motor

#### 3.1.2.1 Tổng quan về DC motor

Động cơ DC được lựa chọn cho hệ thống băng chuyền vì có cấu trúc đơn giản, dễ điều khiển tốc độ và đáp ứng nhanh với sự thay đổi tải. Tốc độ của động cơ DC có thể điều chỉnh trực tiếp thông qua điện áp hoặc độ rộng xung PWM, giúp việc điều khiển băng chuyền trở nên linh hoạt và chính xác. So với các loại động cơ khác như động cơ AC hay động cơ bước, động cơ DC có chi phí thấp, dễ tích hợp với vi điều khiển và đủ mô-men xoắn cho các hệ thống băng chuyền quy mô nhỏ và trung bình.

Bộ điều khiển PID đặc biệt hiệu quả khi áp dụng cho động cơ DC do đặc tính động học gần tuyến tính và mô hình điều khiển đơn giản của loại động cơ này. Mối quan hệ trực tiếp giữa tín hiệu điều khiển và tốc độ giúp PID dễ tinh chỉnh và đạt hiệu quả cao trong việc duy trì tốc độ ổn định. Trong khi đó, các loại động cơ khác thường có đặc tính phi tuyến hoặc đã tích hợp sẵn bộ điều khiển nội bộ, khiến việc áp dụng PID trực tiếp kém hiệu quả hoặc không cần thiết.

**Phần cứng sử dụng:**




Hình 3.3: Động cơ DC giảm tốc V1 Dual Shaft Plastic Geared TT Motor[4]

Động cơ DC giảm tốc V1 Dual Shaft Plastic Geared TT Motor là loại được lựa chọn và sử dụng nhiều nhất hiện nay cho các thiết kế khung Robot, xe, thuyền,... , động cơ có chất lượng và giá thành vừa phải cùng với khả năng dễ lắp ráp đem đến chi phí tiết kiệm và sự tiện dụng cho người sử dụng[4]. Vì thế nhóm chọn động cơ này cho băng chuyền quy mô nhỏ với phân loại hàng hóa nhẹ không cần sức kéo mạnh.Ngoài ra động cơ còn kết hợp được với bánh đĩa 20 xung phục vụ cho quá trình encoder lấy dữ liệu tốc độ phục vụ cho thiết kế Bộ điều khiển PID.

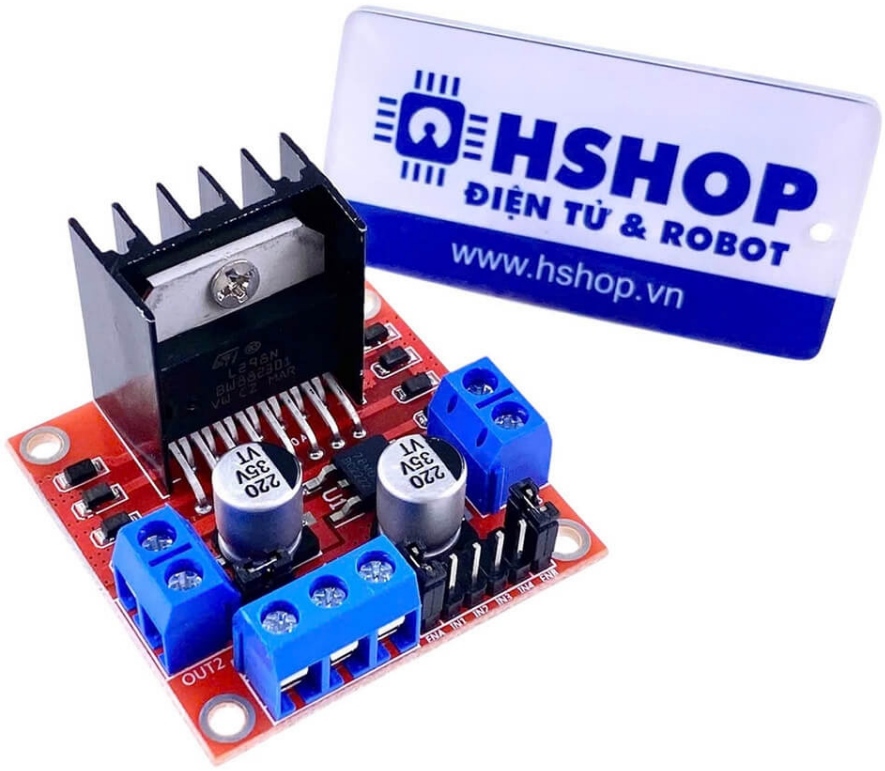
#### 3.1.2.2 Tổng quan về H-brigde

Tuy nhiên, để điều khiển được động cơ DC, không thể cấp trực tiếp nguồn từ vi xử lý vào động cơ do khả năng cấp dòng của vi xử lý rất nhỏ, không đáp ứng được yêu cầu công suất của động cơ. Vì vậy, cần sử dụng mạch H-bridge làm tầng khuếch đại công suất trung gian. H-bridge cho phép vi xử lý đưa tín hiệu điều khiển dạng xung PWM vào mạch công suất, từ đó mạch sẽ cung cấp dòng điện và điện áp phù hợp cho động cơ DC, đồng thời cho phép điều khiển tốc độ và chiều quay một cách linh hoạt

và hiệu quả.

Mạch cầu H là một mạch điện cơ bản dùng để điều khiển động cơ DC quay thuận hoặc quay nghịch bằng cách thay đổi chiều dòng điện cấp vào động cơ. Trong thực tế, mạch cầu H có nhiều cấu hình khác nhau tùy thuộc vào việc lựa chọn linh kiện công suất, khả năng chịu dòng và điện áp, cũng như tần số xung PWM sử dụng, từ đó quyết định hiệu năng và khả năng điều khiển của mạch. Nguyên lý hoạt động của mạch dựa trên đặc tính của động cơ DC: khi đảo chiều dòng điện chạy qua động cơ thì chiều quay của động cơ cũng thay đổi tương ứng. Vì vậy, bằng cách điều khiển các khóa trong mạch cầu H để đổi chiều cấp điện, có thể điều khiển linh hoạt chiều quay và tốc độ của động cơ.

**Phần cứng sử dụng:**




Hình 3.4: Mạch điều khiển động cơ L298 DC Motor Driver[6]

L298 là mạch điều khiển động cơ tích hợp hai cầu H độc lập, cho phép điều khiển linh hoạt hai động cơ DC hoặc một động cơ bước. Mỗi cầu H có khả năng đảo chiều dòng điện để điều khiển chiều quay thuận và nghịch của động cơ, đồng thời có thể kết hợp với tín hiệu PWM để điều chỉnh tốc độ. Mạch L298 DC Motor Driver hỗ trợ dòng tải lên đến khoảng 2 A cho mỗi kênh, được tích hợp diode bảo vệ chống dòng ngược và IC ổn áp 7805 để cung cấp nguồn 5 VDC cho các module điều khiển. Nhờ cấu trúc hai cầu H và khả năng làm việc ổn định, L298 được sử dụng phổ biến trong các hệ thống điều khiển động cơ DC và động cơ bước.

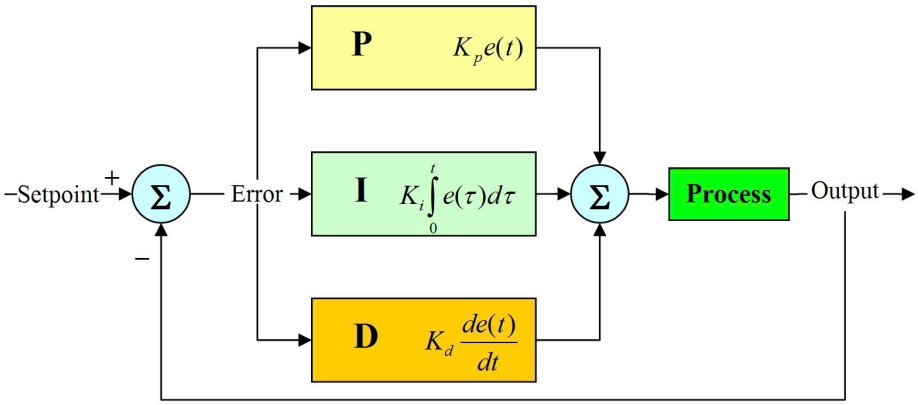
#### 3.1.2.3 PID

**Tổng quan về PID:**

PID (Proportional Integral Derivative hay tỉ lệ - tích phân - vi phân) là giải thuật/ cơ chế feedback vòng điều khiển, nhằm tính toán sai số giữa giá trị đo thông số biến đổi và giá trị đặt mong muốn (output - setpoint), từ đó sẽ giảm thiểu tối đa sai số bằng cách điều chỉnh các giá trị đầu vào[9].
Thông thường, giải thuật PID sẽ gồm 3 thông số cơ bản (hay 3 khâu) đại diện cho các giá trị tỉ lệ, tích phân và đạo hàm (P, I và D):

- Tỉ lệ (P): sẽ tính toán xác định tác động của sai số hiện tại, tức giá trị tín hiệu tỉ lệ với mức sai số thời điểm hiện tại.
- Tích phân (I): sẽ tính toán tích lũy của sai số trong quá khứ, nhằm triệt tiêu các sai số tĩnh (sai số tĩnh ở đây là các sai số nhỏ không đáng kể khiến giá trị hiện tại gần như bằng giá trị mong muốn, khi đó động cơ đã ổn định nên có thể sẽ không điều chỉnh sai số này).
- Vi phân/ đạo hàm (D): tỉ lệ với tốc độ thay đổi của sai số, nhằm dự đoán và điều chỉnh sớm các sai số có thể xảy ra trong tương lai, giảm dao động giá trị.

Tổng 3 giá trị trên cho ra được thừa số tỉ lệ nhằm điều chỉnh thông số đích (và cũng là thông số tạo ra sai số nguồn đầu vào cho 3 thành phần), có thể quan sát hình bên dưới.




Hình 3.5: Flow tính toán PID

**Áp dụng PID vào hệ thống cho DC Motor:**

**Chuẩn bị:**

- Ta sẽ sử dụng đơn vị *RPS* (vòng trên giây) để làm chuẩn cho giá trị tính toán trên PID đối với DC motor hiện tại.
- Nguồn lấy giá trị thực tế để so với setpoint là mạch encoder được gắn sát với đĩa quay encoder (20 lỗ) nhằm đo giá trị *(*RPS) hiện tại của motor.
- Quá trình xử lý tính toán của PID sẽ được thực hiện bởi ESP32, và sẽ được chuyển đổi tương ứng để truyền đi PWM vào L298N để điều khiển tốc độ của motor.

**Quá trình thực hiện:**

- Ta sẽ bắt đầu với encoder bằng việc thực hiện lấy mẫu tốc độ (*RPS*) 83*ms* một lần, và truyền đi giá trị này vào ESP32 để tính toán.

- Khi đã có giá trị tốc độ hiện tại, ta sẽ tìm sai số (error) dựa trên giá trị đó và setpoint, từ đó thực hiện tính toán như sau:

```
float error = target - measured;
if (error > -1.0f && error < 1.0f) {
    error = 0.0f;
    // error nho hon nguong se bi triet tieu.
}
pid->integral += error * dt;
if (pid->integral > 20.0f) pid->integral = 20.0f;
if (pid->integral < -20.0f) pid->integral = -20.0f;

float output =
      pid->kp * error
    + pid->ki * pid->integral
    - pid->kd * (measured - pid->prev_measured) / dt;
output = (output*0.6 + last_delta*0.4);
```

  + kp sẽ được nhân với sai số, ki sẽ nhân với tích phân sai số theo thời gian, kd sẽ được nhân với sai lệch giữa giá trị đo cũ và giá trị đo mới.
  + Tuy nhiên, tích phân sẽ được giới hạn ở -20.0f tới 20.0f để intergral không bị "tích" và tăng quá cao.
  + Cuối cùng, đầu ra output của phép tính sẽ được làm mượt bằng công thức:

    $$\text{final_output} = (\text{output} \times 0.6 + \text{last_output} \times 0.4)$$

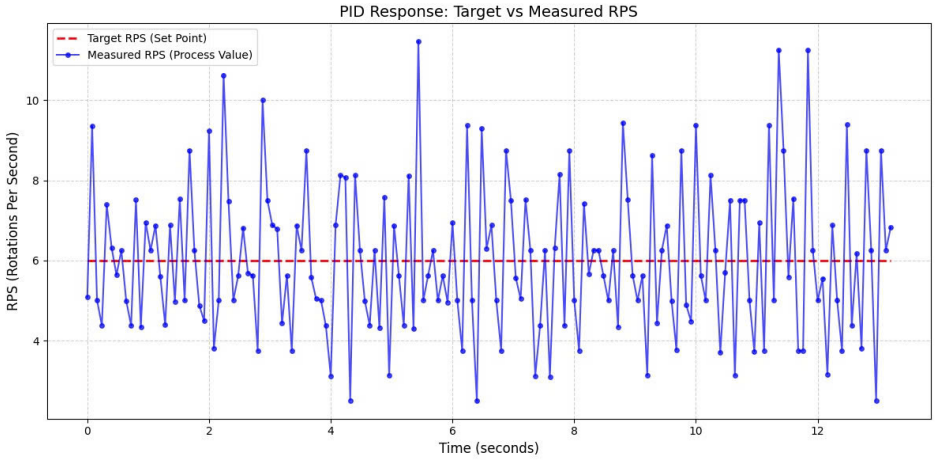
    Điều này sẽ làm giảm sự ảnh hưởng của giá trị đo (measured/error) hiện tại và thêm vào sự ảnh hưởng của giá trị cũ, khiến giá trị sẽ được ổn định hơn.

- Kết thúc quá trình trên ta được giá trị output (delta) cho PID, ta thực hiện cộng với CurrentPWM (PWM hiện tại cho L298N) để điều chỉnh.Tất nhiên, giá trị này sẽ được tiền xử lý trước khi đưa vào cộng thẳng với PWM.
- Sau khi có giá trị của PWM, việc còn lại chỉ là gọi hàm để truyền đi cho L298N.

**Hiển thị giá trị đo lên LCD:**

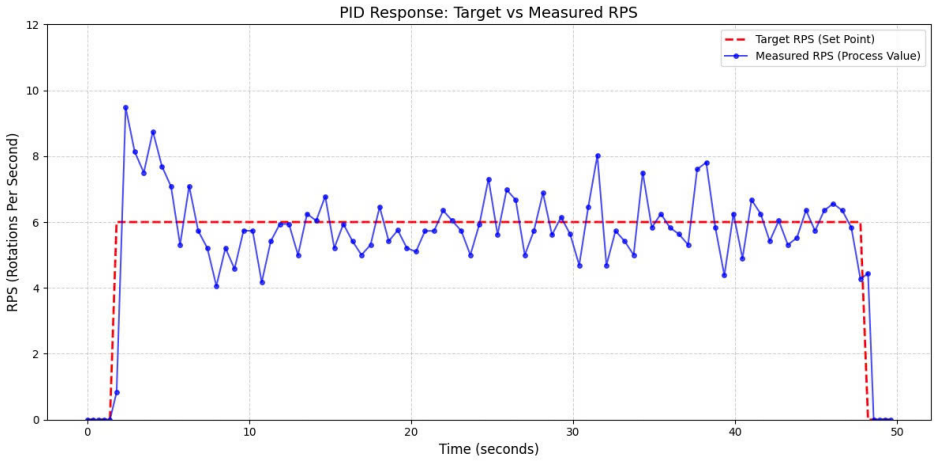
- Qua quá trình thử nghiệm, nhóm thấy không thể hiển thị giá trị *RPS* được đo trong $83ms$ bằng encoder để đại diện cho một giá trị như "vòng trên giây" ($1s$) được, bởi giá trị đo lấy từ encoder khá nhiễu. Nên nhóm quyết định tăng thời gian lấy mẫu (tức số mẫu) và hiển thị lên LCD:
  - Với mỗi mẫu lấy trong $83ms$, ta lấy 6 mẫu như vậy ( $498ms$, coi như $0.5s$) rồi lấy trung bình cộng. Kết quả cho ra ổn định hơn và ít dao động hơn so với việc lấy mẫu với thời gian dưới $100ms$ như cũ.
  - Để cho trực quan hơn, nhóm lấy log đọc được từ ESP32 qua quá trình chạy (AUTO mode) để plot thành graph.

**Cách lấy mẫu cũ (83*ms/sample*):**




**Đánh giá:** Vì tần số lấy mẫu cao nên điểm dữ liệu khá dày đặc, dao động khá nhiều (gần như có lúc gấp đôi giá trị setpoint).

**Cách lấy mẫu mới (500*ms/sample*) và giảm tốc độ thay đổi của delta (giới hạn mỗi step chỉ 0.25%/100% cho dutycycle):**




**Đánh giá:**

- Khi vào giai đoạn ổn định, tốc độ đo được giao động quanh setpoint cỡ 2*RPS*.
- Vì motor phải kéo cả một băng tải dài nên nhóm thấy như này là hoàn toàn chấp nhận được, quan sát thực tế thấy hoạt động của motor khi setpoint trong khoảng 4-10*RPS* hoạt động khá ổn và hoàn toàn bình thường.

### 3.1.3 Các phần cứng chuyên dụng

#### 3.1.3.1 Tổng quan về Color Sensor

Cảm biến màu (color sensor) là một thiết bị quang điện dùng để đo lường và nhận diện màu sắc của vật thể thông qua việc phân tích ánh sáng phản xạ từ bề mặt của nó. Cảm biến thường chiếu ánh sáng lên vật thể, sau đó thu nhận ánh sáng phản xạ và xác định cường độ của các thành phần màu cơ bản như đỏ (R), xanh lá (G) và xanh dương (B). Nhờ đó, cảm biến có thể suy ra màu sắc tổng thể của vật thể. Cảm biến màu được ứng dụng rộng rãi trong các lĩnh vực như robot, tự động hóa công nghiệp, y tế và các hệ thống IoT.

Nguyên lý hoạt động của cảm biến màu dựa trên quá trình phân tích ánh sáng. Ánh sáng từ nguồn chiếu lên vật thể sẽ bị phản xạ hoặc truyền qua và đi vào cảm biến. Bên trong cảm biến có các bộ lọc màu hoặc mảng cảm biến quang học tương ứng với các kênh RGB, mỗi kênh có nhiệm vụ đo cường độ ánh sáng của một màu cơ bản. Các giá trị R, G và B thu được sẽ được vi xử lý trong cảm biến hoặc trong hệ thống điều khiển tổng hợp và xử lý để xác định màu sắc của vật thể.

**Phần cứng sử dụng:**




Hình 3.6: Cảm biến màu sắc $I^2C$ UART TCS34725 Color Sensor V1[1]

Cảm biến màu TCS34725 sử dụng chip TCS34725, cho phép đo và xuất trực tiếp các giá trị màu RGB thông qua giao tiếp $I^2C$ (và một số mô-đun hỗ trợ thêm UART). Cảm biến có thể được sử dụng dưới dạng mô-đun đơn giản hoặc kết hợp với vi điều khiển để đọc và xử lý dữ liệu màu. Trong một số cấu hình, MCU nội bộ có thể thực hiện việc cân bằng trắng và xuất trực tiếp giá trị RGB, giúp giảm tải cho hệ thống điều khiển trung tâm. Nhờ khả năng giao tiếp linh hoạt và dữ liệu đầu ra trực tiếp, TCS34725 dễ dàng tích hợp vào các hệ thống nhúng.

Việc sử dụng TCS34725 mang lại nhiều lợi ích cho hệ thống phân loại màu. Cảm biến được tích hợp bộ lọc chặn hồng ngoại (IR filter), giúp giảm ảnh hưởng của thành phần quang phổ IR trong ánh sáng môi trường, từ đó nâng cao độ chính xác của phép đo màu. Ngoài ra, TCS34725 có khả năng xuất dữ liệu RGB ổn định mà không cần

thực hiện cân bằng trắng phức tạp ở mức ứng dụng, giúp đơn giản hóa thuật toán xử lý và tăng độ tin cậy của hệ thống. Những đặc điểm này khiến TCS34725 trở thành lựa chọn phù hợp cho các ứng dụng nhận diện màu trong robot và hệ thống tự động hóa.

#### 3.1.3.2 Tổng quan về Encoder

Trong các hệ thống điều khiển động cơ DC, vi xử lý chỉ có khả năng phát xung điều khiển (như PWM) để điều chỉnh tốc độ hoặc hướng quay, nhưng không thể trực tiếp nhận biết các thông tin phản hồi về trạng thái hoạt động thực tế của động cơ như vị trí, số vòng quay hay tốc độ. Do đó, encoder trở thành phần tử phản hồi quan trọng, giúp cung cấp dữ liệu cần thiết để vi xử lý giám sát và điều khiển động cơ chính xác hơn, đồng thời hình thành hệ thống điều khiển vòng kín nhằm tăng độ ổn định và khả năng bù sai lệch trong quá trình vận hành.

Encoder hoạt động dựa trên nguyên lý đĩa mã hóa quay quanh trục động cơ, trên đĩa có các rãnh cho phép ánh sáng từ nguồn phát xuyên qua. Khi đĩa quay, cảm biến quang sẽ luân phiên ghi nhận trạng thái có hoặc không có ánh sáng, từ đó tạo ra chuỗi xung vuông. Số xung thu được phản ánh vị trí hoặc số vòng quay, trong khi tần số xung cho biết tốc độ của động cơ. Các tín hiệu xung này được truyền về bộ xử lý trung tâm như vi xử lý hoặc PLC để phục vụ cho việc giám sát và điều khiển.

**Phần cứng sử dụng:**



Hình 3.7: Đĩa Encoder 20 xung [3]




Hình 3.8: Mạch cảm biến tốc độ Encoder V1[7]

Mạch cảm biến tốc độ Encoder V1 bao gồm 1 mắt phát và 1 mắt thu hồng ngoại đặt cách nhau qua 1 khe hở, khi ánh sáng từ mắt phát đi được tới mắt thu (xuyên qua lỗ của dĩa encoder) thì sẽ có tín hiệu mức cao (5v) phát ra khỏi chân out, khi bị che lại

thì chân out phát ra tín hiệu mức thấp (0v). Encoder V1 và đĩa Encoder 20 xung được thiết kế phù hợp để tương thích với Động Cơ DC Giảm Tốc V1 Dual Shaft Plastic Geared TT Motor.

#### 3.1.3.3 Tổng quan về Servo

Servo là một hệ thống truyền động điều khiển hồi tiếp vòng kín, nhận tín hiệu điều khiển từ vi điều khiển (MCU) nhằm đảm bảo góc độ chuyển động chính xác và tốc độ đáp ứng nhanh. Dựa trên tín hiệu đặt từ MCU và tín hiệu phản hồi từ encoder, bộ điều khiển servo liên tục so sánh giá trị mong muốn với trạng thái thực tế để điều chỉnh kịp thời, giúp servo đạt và duy trì đúng góc quay yêu cầu cũng như phản ứng nhanh trước sự thay đổi lệnh hoặc nhiễu tải. Trong đề tài, nhóm ứng dụng servo để xây dựng các cánh tay robot (arm-robot) chuyển động lên xuống thực hiện thao tác gạt và phân loại hàng hóa sau khi đã được nhận diện màu sắc bằng cảm biến màu, qua đó đảm bảo quá trình phân loại diễn ra chính xác, nhanh chóng và đồng bộ.

**Phần cứng sử dụng:**




Hình 3.9: Động cơ RC Servo 9G 0-180°[5]

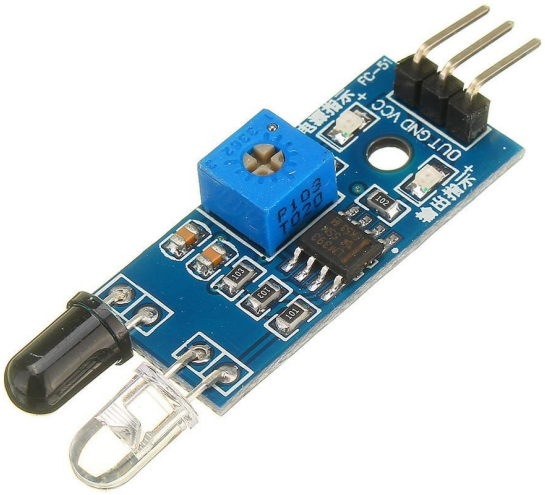
Động cơ RC Servo 9G 0-180° có kích thước nhỏ, là loại được sử dụng nhiều nhất để làm các mô hình nhỏ hoặc các cơ cấu kéo không cần đến lực nặng, động cơ RC

Servo 9G có tốc độ phản ứng nhanh, các bánh răng được làm bằng nhựa nên cần lưu ý khi nâng tải nặng vì có thể làm hư bánh răng, động cơ RC Servo 9G có tích hợp sẵn Driver điều khiển động cơ bên trong nên có thể dễ dàng điều khiển góc quay bằng phương pháp điều độ rộng xung PWM[5].

#### 3.1.3.4 Tổng quan về IR Infrared Obstacle Avoidance

Cảm biến vật cản hồng ngoại sử dụng một cặp truyền và nhận tia hồng ngoại. Tia hồng ngoại phát một tần số nhất định khi gặp vật cản sẽ phản xạ vào đèn thu hồng ngoại, sau khi qua IC so sánh đèn màu xanh sẽ sáng lên, đồng thời cho tín hiệu số đầu ra. Vì thế cảm biến vật cản hồng ngoại thường được ứng dụng nhiều trong thực tế như xe tự hành, băng chuyền phân loại,.. Và nhóm sử dụng cảm biến để có thể nhận diện hàng đã rơi vào ô phân loại phục vụ cho việc đếm và kiểm tra số lượng hàng đã được phân loại.

**Phần cứng sử dụng:**




Hình 3.10: Cảm biến vật cản hồng ngoại IR Infrared Obstacle Avoidance[2]

Cảm biến vật cản hồng ngoại IR Infrared Obstacle Avoidance được sử dụng để

nhận biết vật cản bằng ánh sáng hồng ngoại, cảm biến có cách sử dụng đơn giản với biến trở chỉnh khoảng cách nhận biết vật cản, ngõ ra dạng Digital dễ dàng giao tiếp và lập trình với Vi điều khiển, thích hợp để làm các ứng dụng Robot tránh vật cản, báo trộm, mô hình cửa tự động.

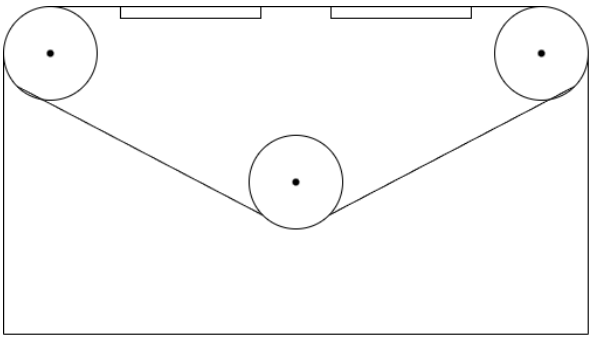
#### 3.1.3.5 Tổng quan về băng chuyền

**Ý tưởng thiết kế:**



Hình 3.11: Mô hình băng chuyền lần đầu.

Ban đầu, nhóm dự định thiết kế một băng chuyền có chiều dài khoảng 60 *cm*, phù hợp cho việc phân loại ba màu (Red – Green – Blue), đảm bảo khoảng cách giữa các vị trí phân loại không quá gần, đồng thời tổng thể hệ thống không quá dài để thuận tiện cho việc di chuyển và lắp đặt. Băng chuyền được thiết kế với hai trục quay đặt ở hai đầu, trong đó một trục được nối với động cơ DC để tạo chuyển động quay. Bề mặt băng chuyền sử dụng dây thun màu đen có chiều rộng 10 *cm*, nhằm giúp cảm biến màu (color sensor) nhận diện màu sắc ổn định và chính xác hơn.



Hình 3.12: Mô hình băng chuyền sau khi chỉnh sửa.

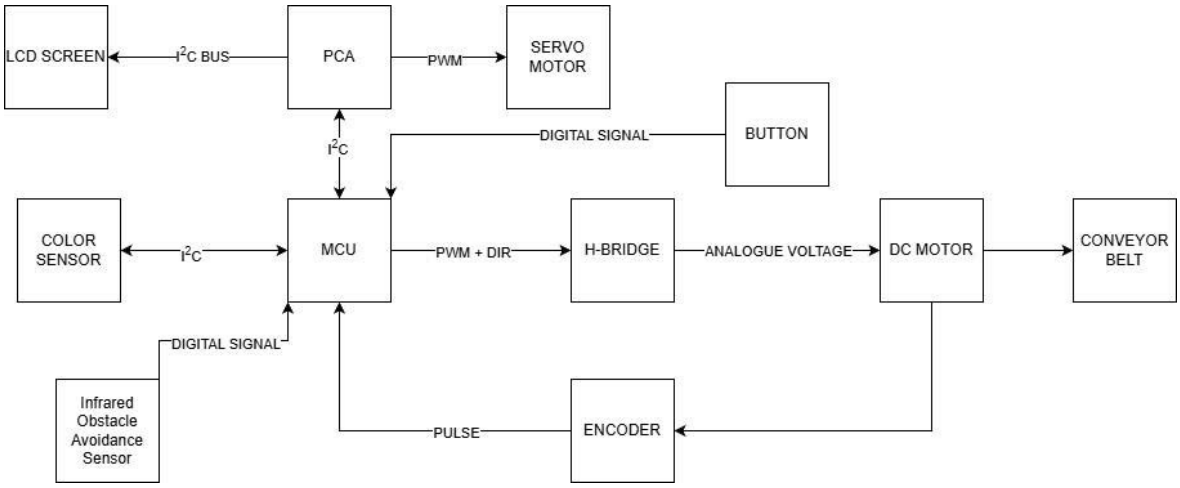
Tuy nhiên, trong quá trình thực hiện, nhóm nhận thấy thiết kế ban đầu tồn tại một số nhược điểm nên đã tiến hành điều chỉnh. Cụ thể, cấu hình chỉ sử dụng hai trục quay tuy có thể vận hành nhưng không tạo đủ lực căng, khiến dây thun khó bám sát vào trục quay, làm cho băng chuyền hoạt động kém ổn định. Do đó, nhóm đã bổ sung thêm một trục quay ở vị trí phía dưới, nằm giữa hai trục chính, tạo thành cấu trúc dạng tam giác. Việc bổ sung trục này giúp tăng lực căng cho dây thun, đảm bảo dây bám chặt vào các trục quay chính và giúp băng chuyền vận hành ổn định hơn. Ngoài ra, nhóm còn thiết kế thêm hai mặt đỡ bên dưới để giảm độ lún ở khu vực giữa băng chuyền, giúp vật đặt trên bề mặt băng chuyền được giữ vững trong suốt quá trình di chuyển.

## 3.2 Phương pháp nghiên cứu

### 3.2.1 Thiết kế sơ đồ hệ thống (System diagram)

Sau khi nghiên cứu và tổng hợp các cơ sở lý thuyết liên quan, nhóm tiến hành xây dựng phương pháp nghiên cứu cho hệ thống. Trong giai đoạn này, nhóm thiết kế sơ đồ hệ thống (system diagram) nhằm mô tả tổng quan cấu trúc của hệ thống, thể hiện rõ các khối chức năng chính, các đường truyền tín hiệu, giao thức giao tiếp cũng như mối quan hệ giữa vi xử lý và các phần cứng ngoại vi.

Việc xây dựng system diagram giúp nhóm có cái nhìn tổng thể về hệ thống ngay từ giai đoạn thiết kế, đảm bảo sự liên kết chặt chẽ giữa các thành phần phần cứng và phần mềm, đồng thời làm cơ sở cho việc triển khai chi tiết các sơ đồ khối, static diagram và sequence diagram ở các phần tiếp theo. Sơ đồ hệ thống cũng góp phần hạn chế sai sót trong quá trình thi công và lập trình, đồng thời hỗ trợ việc mở rộng và bảo trì hệ thống trong tương lai.




Hình 3.13: Tổng quan sơ đồ hệ thống.

#### 3.2.1.1 Các khối thành phần và vai trò

- MCU (Microcontroller):
  - + Vai trò: bộ điều phối trung tâm, thực thi thuật toán phân loại màu, điều khiển PID cho tốc độ băng, phát tín hiệu điều khiển servo/driver, ghi/hiển thị trạng thái.
  - + Giao tiếp: $I^2C$ (với color sensor, có thể LCD), digital inputs (button, IR), PWM outputs (servo, driver motor), interrupts/timers (encoder).
- Color Sensor (TCS34725)
  - + Vai trò: đọc giá trị màu (RGB) của vật đi qua.
  - + Giao tiếp: $I^2C$ hai chiều với MCU. MCU cấu hình sensor và đọc dữ liệu. Dữ liệu này quyết định hành động phân loại.
- Màn hình LCD
  - + Vai trò: Hiển thị kết quả phân loại sản phẩm, tốc độ băng tải và trạng thái hoạt động hiện tại của hệ thống.

+ Giao tiếp: Màn hình LCD được điều khiển bởi ESP32 thông qua giao tiếp $I^2C$, cho phép hiển thị thông tin một cách ổn định và giảm số lượng chân GPIO cần sử dụng trên vi điều khiển.

- Infrared Obstacle/Avoidance Sensor
  + Vai trò: phát hiện có vật hoặc chướng ngại trên băng (đếm sản phẩm, tránh kẹt).
  + Giao tiếp: tín hiệu digital (GPIO input) tới MCU dùng để kích hoạt đọc màu, bắt đầu/ dừng băng.
- Button
  + Vai trò: điều khiển người dùng (start/stop, tăng/giảm tốc độ, chế độ).
  + Giao tiếp: digital input tới MCU đã được debounce.
- PCA9685
  + Vai trò: bộ mở rộng PWM cho servo khi MCU không đủ kênh PWM.
  + Giao tiếp: $I^2C$ với MCU và xuất đầu ra PWM tới Servo.
- Servo Motor
  + Vai trò: cơ cấu chọn/chặn/đẩy sản phẩm (cơ cấu phân loại).
  + Giao tiếp: PWM (từ MCU hoặc PCA).
- Motor Driver (H-Bridge)
  + Vai trò: khuếch đại tín hiệu điều khiển để cấp nguồn cho DC motor.
  + Giao tiếp: nhận PWM từ MCU từ đó xuất analogue voltage tới DC motor.
- DC Motor
  + Vai trò: kéo băng chuyền.

+ Giao tiếp: nhận công suất từ motor driver; cơ khí truyền động đến conveyor belt; đồng thời nối với encoder để cung cấp feedback.

- Encoder Sensor

  + Vai trò: cung cấp xung (pulse) / quadrature để đo tốc độ hoặc vị trí trục motor (feedback cho PID).

  + Giao tiếp: pulse/quadrature MCU (thường qua interrupts / timer capture).

- Conveyor Belt

  + Vai trò: băng vận chuyển sản phẩm, chịu tác động cơ học bởi DC motor.

#### 3.2.1.2 Các mối quan hệ chính

**1. Vòng điều khiển tốc độ băng chuyền (closed-loop control)**

Trong hệ thống, tốc độ băng chuyền được điều khiển theo nguyên lý vòng kín nhằm đảm bảo băng chuyền luôn hoạt động ổn định theo giá trị mong muốn. Cụ thể, MCU phát tín hiệu PWM đến mạch điều khiển động cơ (motor driver). Mạch này có nhiệm vụ khuếch đại tín hiệu điều khiển để cấp công suất phù hợp cho động cơ DC. Động cơ DC quay và truyền chuyển động cơ học sang băng chuyền.

Đồng thời, encoder gắn trên trục động cơ tạo ra các xung phản hồi tương ứng với tốc độ quay thực tế của động cơ. Các xung này được đưa về MCU, tại đây MCU so sánh tốc độ thực tế với tốc độ đặt trước và sử dụng thuật toán PID để điều chỉnh lại tín hiệu PWM. Nhờ đó, hệ thống có thể duy trì tốc độ băng chuyền ổn định ngay cả khi có tải thay đổi.

**2. Hoạt động phân loại sản phẩm theo sự kiện (event-driven)**

Quá trình phân loại sản phẩm được xảy ra khi MCU nhận dữ liệu màu sắc từ cảm biến màu thông qua giao tiếp I²C. Dựa trên giá trị màu thu được, MCU thực hiện so

sánh và xác định loại sản phẩm theo màu.

Sau khi hoàn tất quá trình phân loại, MCU điều khiển servo motor bằng tín hiệu PWM (trực tiếp hoặc thông qua mạch PCA) để thực hiện thao tác cơ khí tương ứng, chẳng hạn như đẩy sản phẩm sang nhánh phân loại phù hợp hoặc thay đổi hướng di chuyển của sản phẩm trên băng chuyền.

### 3. Cấu hình và trao đổi dữ liệu trong hệ thống

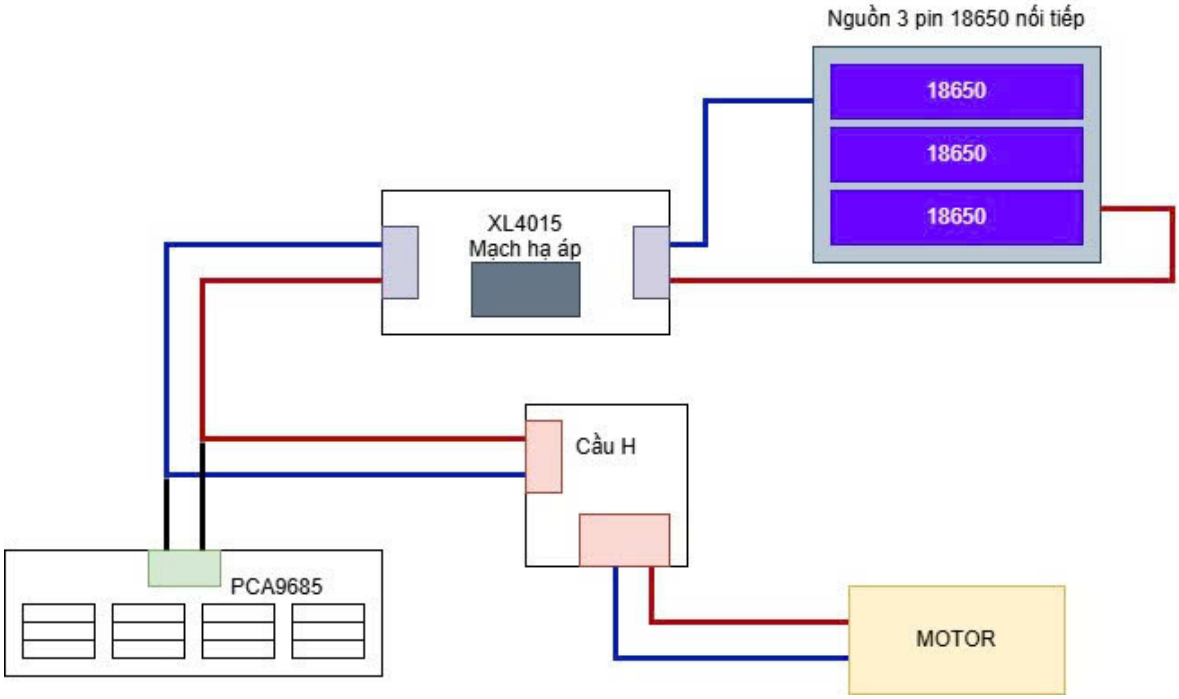
MCU đóng vai trò trung tâm trong việc cấu hình và thu thập dữ liệu từ các thiết bị ngoại vi. Thông qua giao tiếp I²C, MCU cấu hình cảm biến màu và đọc các giá trị RGB trong quá trình vận hành.

Ngoài ra, MCU gửi dữ liệu trạng thái hệ thống, thông tin vận hành hoặc cảnh báo lỗi lên màn hình LCD để người vận hành dễ dàng theo dõi. Nút nhấn được sử dụng như một giao diện điều khiển đơn giản, cho phép người dùng thực hiện các thao tác như khởi động, dừng hay thay đổi chế độ, tốc độ của hệ thống.

### 4. Phân tách giữa khối điều khiển và khối công suất

Trong thiết kế hệ thống, phần điều khiển và phần công suất được tách biệt rõ ràng nhằm đảm bảo an toàn và độ tin cậy.

MCU chỉ làm việc với các tín hiệu mức logic (PWM, tín hiệu số), không trực tiếp cấp nguồn cho động cơ. Toàn bộ dòng điện và công suất lớn cần thiết cho động cơ DC (thông qua cầu H) và Servo (thông qua PCA9685) được đảm nhiệm bởi module hạ áp XL4015 DC-DC. Cách phân tách này giúp bảo vệ MCU khỏi các tác động của dòng điện lớn và nhiễu điện từ do động cơ gây ra, đồng thời nâng cao độ ổn định và tuổi thọ của hệ thống.




Hình 3.14: Nguồn cấp cho cơ cấu chấp hành của hệ thống.

**Đối với nguồn cấp cho actuators như sơ đồ trên:**

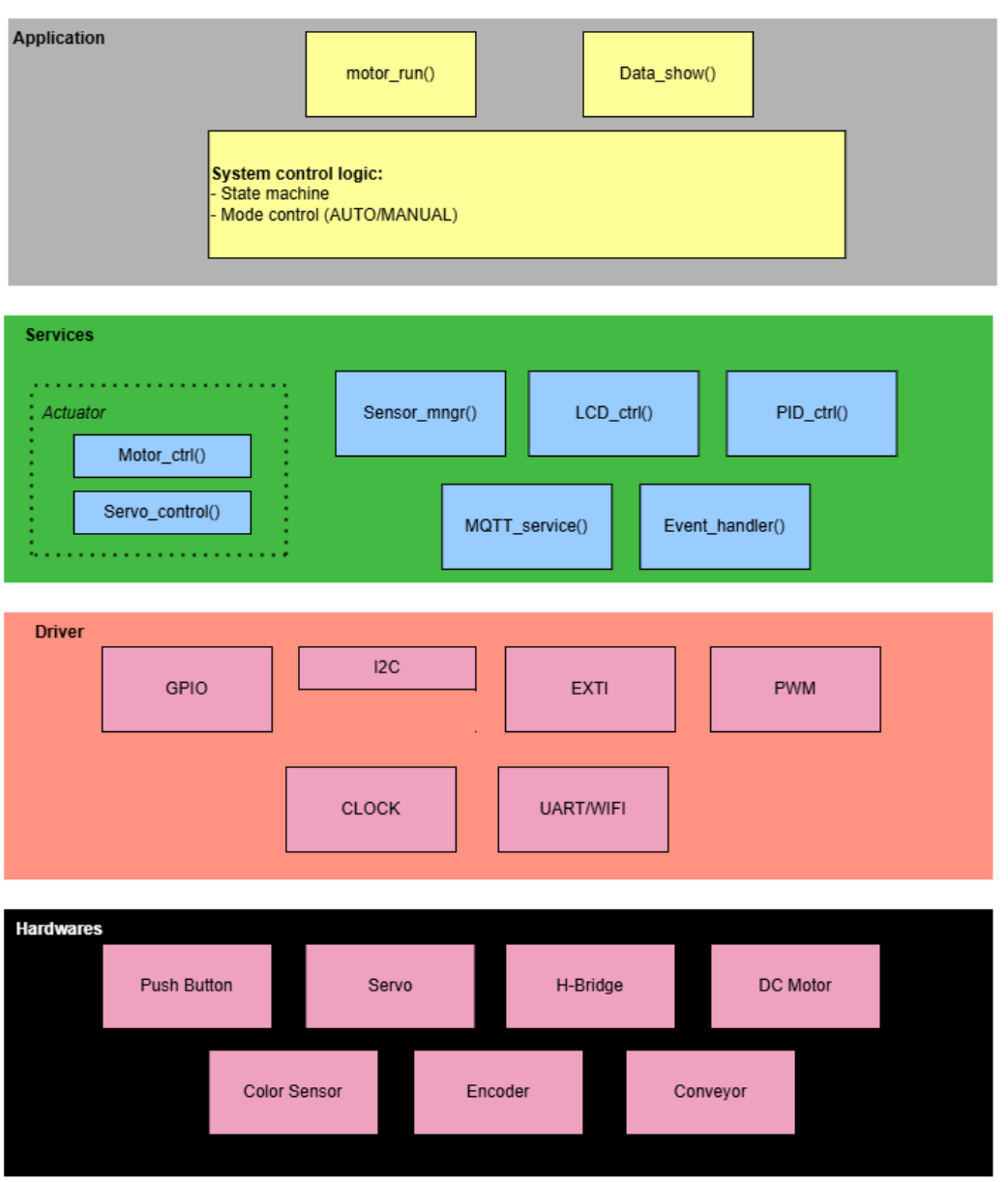
- XL4015 (Mạch hạ áp) đang là nguồn chính, sau đó tách làm 2 nhánh: 1 nhánh vào PCA9685 để cấp nguồn riêng cho servo và 1 nhánh vào cầu H để cấp nguồn cho motor.
- Sử dụng 3 pin 18650 ( 3.6V-4V) suy ra là 11V đến 12V (có thể chỉ có 9V khi xả). Tuy nhiên đi qua mạch hạ áp nên sẽ tất cả sẽ phụ thuộc vào áp đầu ra của mạch hạ áp.
- Điện áp hoạt động của servo là dưới 6V và motor là 4V-9V nên nhóm quyết định chọn áp đầu ra của mạch hạ áp rơi vào giữa 5V-6V(5.5V).
- Tuy nhiên, khi sử dụng L298N có thể gây sụt áp và đồng thời là hiện tượng deadzone ở L298N + Motor (áp cấp qua PWM không đủ để chạy motor hoặc thắng được ma sát dừng của motor với băng tải), vì vậy điều này sẽ được giải

quyết bằng phần mềm là đặt ra một ngưỡng PWM_DeadZone khi điều khiển tốc độ motor (nhóm dùng 30%) để làm MIN cho PWM khi tính toán PID.

### 3.2.2 Sơ đồ khối của hệ thống (Block diagram)

Sau khi đã xây dựng hoàn chỉnh System Diagram để mô tả tổng thể hệ thống và luồng tương tác giữa các thành phần, sơ đồ khối (Block diagram) được triển khai nhằm phân rã hệ thống thành các khối chức năng rõ ràng. Block diagram tập trung vào việc tổ chức hệ thống theo từng tầng ứng dụng, làm nổi bật vai trò và phạm vi trách nhiệm của mỗi khối thay vì đi sâu vào chi tiết triển khai.

Mục đích của block diagram là giúp làm rõ mối quan hệ giữa các tầng trong hệ thống, từ tầng phần cứng, tầng driver, tầng dịch vụ cho đến tầng ứng dụng, qua đó giảm sự phụ thuộc trực tiếp giữa các thành phần và tăng tính mô-đun trong thiết kế. Sơ đồ này đóng vai trò như một cầu nối giữa system diagram và thiết kế chi tiết, hỗ trợ việc phân công chức năng, phát triển phần mềm, bảo trì và mở rộng hệ thống một cách hiệu quả.




Hình 3.15: Tổng quan sơ đồ khối hệ thống.

#### 3.2.2.1 Vai trò và thành phần của từng tầng

**1. Tầng ứng dụng ( Application layer )**

Application layer là tầng điều khiển mức cao nhất, chịu trách nhiệm quản lý logic tổng thể của hệ thống, điều phối các chế độ hoạt động như MANUAL/AUTO và quyết định xem khi nào gọi các services tầng bên dưới.
Các thành phần trong Application:

- System control logic
  + State machine: Quản lý các trạng thái hệ thống.
  + Mode Control (Auto/Manual): Điều khiển hai chế độ hoạt động của băng chuyền. Khi hệ thống hoạt động ở chế độ Manual, bằng chuyền sẽ ở trạng thái nghỉ do color sensor bị block không đọc dữ liệu, chỉ chạy khi người dùng đặt sản phẩm và ấn nút phân loại, tốc độ sẽ được điều chỉnh các mức khác nhau bằng bộ điều khiển PID theo màu sản phẩm. Còn khi ở chế độ Auto, băng chuyền duy trì tốc độ chậm và color sensor luôn mở để đọc dữ liệu, phát hiện và phân loại tự động.
- motor_run()
  + Yêu cầu hệ thống cho phép motor chạy hoặc dừng.
  + Gọi các service điều khiển motor tương ứng.
  + Không tạo PWM trực tiếp.
  + Chỉ gọi Motor_ctrl() ở tầng Services.
- Data_show()
  + Hàm hiển thị các thông tin của hệ thống như: Tốc độ hệ thống, kết quả phân loại.
  + Gọi lcd_ctrl() ở tầng Services.
  + Không giao tiếp LCD trực tiếp.

**2. Tầng dịch vụ ( Services layer )**

Tầng Services đóng vai trò là một tầng trung gian giữa Driver và Application thực hiện các hành động như: xử lý nghiệp vụ cụ thể, che giấu chi tiết Driver và cung cấp API rõ ràng cho Application.
Các thành phần trong tầng Services:

- Motor_ctrl()
  + Điều khiển động cơ DC: Nhận yêu cầu tốc độ từ Application, kết hợp với PID controller để xuất giá trị cho PWM driver.
  + Không quan tâm băng đang ở Auto hay Manual.
- Servo_ctrl()
  + Điều khiển servo motor để chắn sản phẩm xuống ô phân loại.
  + Nhận lệnh phân loại từ Application.
  + Chuyển đổi lệnh phân loại thành giá trị PWM phù hợp và sử dụng PWM driver để điều khiển servo.
- Sensor_mngr()
  + Quản lý tất cả cảm biến như: đọc cảm biến màu (qua $I^2C$), đọc encoder (xung phản hồi), đọc cảm biến hồng ngoại.
  + Chuẩn hóa dữ liệu để Application dễ sử dụng.
  + Không đưa dữ liệu thô lên Application.
- PID_ctrl()
  + Thực hiện thuật toán PID: So sánh tốc độ đặt và tốc độ thực tế.
  + Trả kết quả cho Motor_ctrl().
- LCD_ctrl()

+ Đóng gói các thao tác hiển thị: ghi dữ liệu, cập nhật nội dung LCD.
+ Application chỉ gọi hàm hiển thị, không quan tâm giao thức.

- MQTT_service()
  + Thực hiện truyền dữ liệu lên hệ thống giám sát: Kết quả phân loại, trạng thái hệ thống, chế độ Auto/Manual.
  + Quản lý kết nối MQTT.
  + Application chỉ quyết định khi nào gửi.
- Event_handler()
  + Nhận các sự kiện từ driver như: nút bấm, ngắt encoder.
  + Chuyển sự kiện phần cứng thành sự kiện logic.
  + Gửi sự kiện cho Application xử lý.

**3. Tầng driver ( Driver layer )**

Driver layer cung cấp các hàm giao tiếp phần cứng mức thấp, không chứa logic nghiệp vụ.
Các thành phần trong tầng Driver:

- GPIO: Đọc / ghi tín hiệu số (nút bấm, cảm biến).
- EXTI: Xử lý ngắt ngoài (button, encoder).
- PWM: Tạo tín hiệu PWM cho motor và servo.
- $I^2C$: Giao tiếp với cảm biến màu, LCD.
- WiFi: Truyền dữ liệu cho MQTT.
- CLOCK: Cấu hình xung nhịp hệ thống.

**4. Tầng phần cứng ( Hardware layer )**

Hardware layer bao gồm các phần tử vật lý của hệ thống, không có logic phần mềm.
Các thành phần trong tầng Hardware:

- Push Button: Giao diện điều khiển người dùng.
- Color Sensor: Nhận dạng màu sản phẩm.
- Encoder: Phản hồi tốc độ động cơ.
- H-Bridge: Khuếch đại công suất cho DC motor.
- PCA9685: Điều khiển servo.
- DC Motor: Kéo băng chuyền.
- Servo: Phân loại sản phẩm.
- Conveyor: Hệ cơ khí vận chuyển sản phẩm.

### 3.2.3 Sơ đồ tĩnh của hệ thống ( Static diagram )

Static diagram được xây dựng nhằm mô tả cấu trúc phần mềm tĩnh của hệ thống sau khi đã xác định kiến trúc tổng thể bằng block diagram. Sơ đồ này giúp làm rõ cách phân tầng phần mềm, mối quan hệ giữa các module và các API được cung cấp và sử dụng ở từng tầng, từ đó đảm bảo tính module hóa, dễ bảo trì và thuận tiện cho việc phát triển, kiểm thử và mở rộng hệ thống trong quá trình thực hiện đồ án.

#### 3.2.3.1 Các module, API được sử dụng ở từng tầng.

**1. Driver Layer**

- CLOCK:

+ esp_timer_get_time(): Hàm trả về giá trị thực của bộ định thời hệ thống ESP32 có độ phân giải cao. Giá trị đó thường được dùng để đo thời gian, xử lý timeout, và thuật toán điều khiển như PID.

- GPIO:
  + gpio_config(): Cấu hình chân GPIO bao gồm hướng, cạnh lên/ cạnh xuống của trở, và chế độ ngắt.
  + gpio_get_level(): Đọc mức logic input hiện tại của chân GPIO.
  + gpio_set_level(): Thiết lập mức logic cho chân GPIO để được cấu hình là ngõ ra.
- I2C:
  + i2c_param_config(): Cấu hình các tham số của ngoại vi I2C như chế độ hoạt động, chân kết nối, và tốc độ xung clock.
  + i2c_driver_install(): Cài đặt driver I2C và cấp phát các tài nguyên cần thiết cho ngoại vi I2C.
  + i2c_master_read_from_device(): Đọc dữ liệu từ thiết bị I2C slave khi hoạt động ở chế độ master.
  + i2c_master_write_read_device(): Thực hiện thao tác ghi và đọc kết hợp tới một thiết bị I2C slave.
  + i2c_cmd_link_create(): Tạo I2C command link dùng để xây dựng một chuỗi các thao tác I2C.
  + i2c_master_start(): Thêm điều kiện START vào chuỗi lệnh I2C.
  + i2c_master_write_byte(): Ghi một byte dữ liệu lên bus I2C như một phần của chuỗi lệnh I2C.
  + i2c_master_stop(): Thêm điều kiện STOP vào chuỗi lệnh I2C.
  + i2c_master_cmd_begin(): Bật hoặc tắt xung clock cho module FTM (Flex-Timer Module).

+ i2c_cmd_link_delete(): Xóa I2C command link và giải phóng các tài nguyên đã được cấp phát.

- PWM:
  + ledc_timer_config(): Cấu hình bộ định thời LEDC PWM dùng để tạo tín hiệu PWM.
  + ledc_channel_config(): Cấu hình kênh PWM LEDC và liên kết kênh đó với một chân GPIO và một bộ định thời (timer) cụ thể. Hàm này phải được gọi sau khi đã cấu hình LEDC timer.
  + ledc_set_duty(): Thiết lập duty cycle của PWM cho một kênh LEDC cụ thể.
  + ledc_update_duty(): Áp dụng chu kỳ công tác (duty cycle) đã được thiết lập trước đó bằng ledc_set_duty() vào phần cứng PWM đầu ra. Hàm này cập nhật tín hiệu PWM thực tế trên chân GPIO.
- EXTI:
  + gpio_install_isr_service(): Cài đặt dịch vụ ngắt GPIO toàn cục, cho phép sử dụng các trình xử lý ngắt GPIO. Hàm này phải được gọi một lần trước khi đăng ký các GPIO ISR handler riêng lẻ.
  + gpio_isr_handler_add(): Đăng ký trình phục vụ ngắt (Interrupt Service Routine – ISR) cho một chân GPIO cụ thể
  + gpio_isr_handler_remove(): Gỡ bỏ trình xử lý ngắt (interrupt handler) đã được liên kết với một chân GPIO.
  + esp_event_loop_create_default(): Tạo vòng lặp sự kiện hệ thống mặc định dùng cho việc phân phối sự kiện bất đồng bộ.
  + Vòng lặp sự kiện này là bắt buộc để xử lý các sự kiện hệ thống như WiFi, IP và MQTT.

+ esp_event_handler_instance_register(): Đăng ký hàm callback xử lý sự kiện vào vòng lặp sự kiện mặc định cho một event base và event ID cụ thể. Hàm xử lý sẽ được thực thi bất đồng bộ khi sự kiện xảy ra. Mặc định, nếu không đăng ký handler, hệ thống không thực hiện hành động nào.

- WIFI:
  + esp_netif_init(): Khởi tạo hệ thống network interface (TCP/IP stack) cho ESP-IDF, là bước bắt buộc trước khi sử dụng WiFi hoặc các giao thức mạng.
  + esp_netif_create_default_wifi_sta(): Tạo và cấu hình network interface mặc định cho WiFi ở chế độ Station (STA).
  + esp_wifi_init(): Tạo và cấu hình network interface mặc định cho WiFi ở chế độ Station (STA).
  + esp_wifi_set_mode(): Thiết lập chế độ hoạt động của WiFi (Station, Access Point hoặc kết hợp).
  + esp_wifi_set_config(): Cấu hình các tham số WiFi như SSID, mật khẩu và chế độ bảo mật.
  + esp_wifi_start(): Khởi động WiFi driver sau khi đã cấu hình xong chế độ và tham số kết nối.
  + esp_wifi_connect(): Thực hiện kết nối tới Access Point đã được cấu hình.
  + esp_mqtt_client_init(): Khởi tạo MQTT client với các thông tin broker, xác thực và cấu hình kết nối.
  + esp_mqtt_client_register_event(): Đăng ký hàm xử lý sự kiện cho MQTT client (kết nối, ngắt kết nối, lỗi, . . . ).
  + esp_mqtt_client_start(): Khởi động MQTT client và bắt đầu quá trình kết nối tới broker.
  + esp_mqtt_client_publish(): Gửi dữ liệu (message) lên MQTT broker theo topic đã chỉ định.

**2. Services Layer**

- Event_handler :
    + button_task_handler(): Xử lý các sự kiện nút nhấn từ người dùng, nhận tín hiệu từ ngắt GPIO và cập nhật trạng thái hệ thống như chuyển chế độ Manual/Auto hoặc yêu cầu Start.
- MQTT_service:
    + wifi_init_sta(): Khởi tạo WiFi ở chế độ Station và kết nối ESP32 tới mạng WiFi được cấu hình.
    + mqtt_app_start(): Khởi tạo và khởi động MQTT client, thiết lập kết nối tới MQTT broker và đăng ký các sự kiện liên quan.
    + send_data_to_mqtt(): Khởi tạo và khởi động MQTT client, thiết lập kết nối tới MQTT broker và đăng ký các sự kiện liên quan.
- PID_ctrl:
    + motor_speed_pid_step(): Thực hiện một bước điều khiển PID để điều chỉnh tốc độ động cơ dựa trên tốc độ đo được từ encoder và giá trị tốc độ mong muốn.
- LCD_ctrl:
    + lcd_init(): Khởi tạo màn hình LCD sử dụng giao tiếp I2C và chuẩn bị cho quá trình hiển thị dữ liệu.
    + lcd_send_cmd(): Khởi tạo màn hình LCD sử dụng giao tiếp I2C và chuẩn bị cho quá trình hiển thị dữ liệu.
    + lcd_send_data(): Gửi dữ liệu ký tự tới LCD để hiển thị.
    + lcd_send_string(): Hiển thị một chuỗi ký tự lên LCD tại vị trí con trỏ hiện tại.

    + lcd_put_cur(): Hiển thị một chuỗi ký tự lên LCD tại vị trí con trỏ hiện tại.
    + lcd_clear(): Xóa toàn bộ nội dung hiển thị trên LCD.

- Sensor_mngr:
    + ir_sensor_init(): Khởi tạo cảm biến hồng ngoại, cấu hình GPIO và ngắt để phát hiện vật thể.
    + ir_sensor_deinit(): Hủy khởi tạo cảm biến hồng ngoại và giải phóng các tài nguyên liên quan.
    + ir_sensor_counter(): Xử lý sự kiện từ cảm biến hồng ngoại và cập nhật số lượng sản phẩm đi qua.
    + tcs34725_init(): Khởi tạo cảm biến màu TCS34725 thông qua giao tiếp I2C.
    + tcs34725_deinit(): Hủy khởi tạo cảm biến màu và giải phóng tài nguyên.
    + tcs_write(): Ghi dữ liệu cấu hình vào các thanh ghi của cảm biến màu TCS34725.
    + tcs_read(): Đọc dữ liệu từ các thanh ghi của cảm biến màu TCS34725.
    + tcs34725_reader(): Thực hiện quá trình đọc dữ liệu màu và cập nhật giá trị RGB bên trong handler.
    + get_rgb_values(): Lấy giá trị RGB hiện tại từ cảm biến màu để phục vụ phân loại sản phẩm.
    + encoder_init(): Khởi tạo encoder, cấu hình GPIO và ngắt để đo tốc độ quay.
    + encoder_get_rps(): Tính toán và trả về tốc độ quay của động cơ theo đơn vị *vòng/giây* (RPS).
    + encoder_ge_rpm(): Tính toán và trả về tốc độ quay của động cơ theo đơn vị *vòng/phút* (RPM).

- Servo_ctrl:
    + pca9685_init(): Khởi tạo IC điều khiển PWM PCA9685 dùng để điều khiển servo.
    + pca9685_set_pwm_freq(): Thiết lập tần số PWM cho PCA9685
    + pca9685_set_pwm(): Thiết lập giá trị PWM cho một kênh cụ thể của PCA9685.
    + servo_open(): Điều khiển servo mở cửa phân loại sản phẩm.
    + servo_close(): Điều khiển servo đóng cửa phân loại sản phẩm.
- Motor_ctrl:
    + motor_init(): Khởi tạo driver động cơ, cấu hình GPIO và PWM cho điều khiển động cơ.
    + motor_set_direction(): Thiết lập chiều quay của động cơ (thuận, nghịch hoặc dừng).
    + motor_set_speed(): Điều chỉnh tốc độ động cơ thông qua tín hiệu PWM.
    + motor_stop(): Dừng động cơ một cách an toàn bằng cách đưa tốc độ về 0 và tắt điều khiển.

**3. Application Layer**

- Data_show:
    + lcd_draw_once(): Khởi tạo và hiển thị nội dung ban đầu trên LCD, bao gồm các thông tin tĩnh như nhãn hiển thị và giá trị khởi tạo.
    + lcd_show_counters(): Cập nhật và hiển thị số lượng sản phẩm đã phân loại theo từng màu lên LCD.
    + speed_monitor_task(): Task theo dõi tốc độ băng chuyền dựa trên dữ liệu encoder và cập nhật thông tin tốc độ lên LCD hoặc log hệ thống theo chu kỳ.

+ mqtt_publish_task(): Task định kỳ gửi dữ liệu hệ thống (số lượng sản phẩm, tốc độ băng chuyền, . . . ) lên MQTT broker để giám sát từ xa.

- Motor_run:

  + main_task(): Task điều khiển chính của hệ thống, chịu trách nhiệm quản lý luồng hoạt động tổng thể, xử lý chế độ Manual/Auto và phối hợp các service như cảm biến, servo và động cơ.

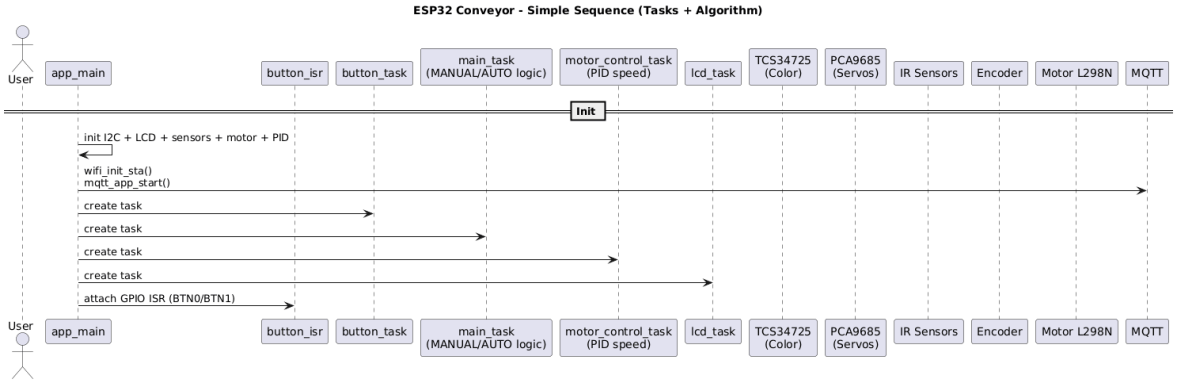
  + button_task(): Task xử lý sự kiện nút nhấn từ người dùng, nhận tín hiệu từ ngắt GPIO và cập nhật trạng thái điều khiển như chuyển chế độ hoạt động hoặc yêu cầu bắt đầu chạy.

### 3.2.4 Thiết kế biểu đồ tuần tự (Sequence diagram)

Sequence diagram là một loại biểu đồ trong UML được sử dụng để mô tả trình tự tương tác giữa các thành phần trong hệ thống theo trục thời gian. Biểu đồ này thể hiện rõ các đối tượng hoặc module tham gia, các lời gọi hàm, tín hiệu hoặc sự kiện được trao đổi, cũng như thứ tự thực hiện của chúng trong một kịch bản hoạt động cụ thể. Mục đích của sequence diagram là giúp làm rõ cách hệ thống vận hành trong từng tình huống, từ đó hỗ trợ việc phân tích luồng xử lý, kiểm tra tính đúng đắn của thiết kế và đảm bảo sự phối hợp nhất quán giữa các tầng như Application, Service và Driver.

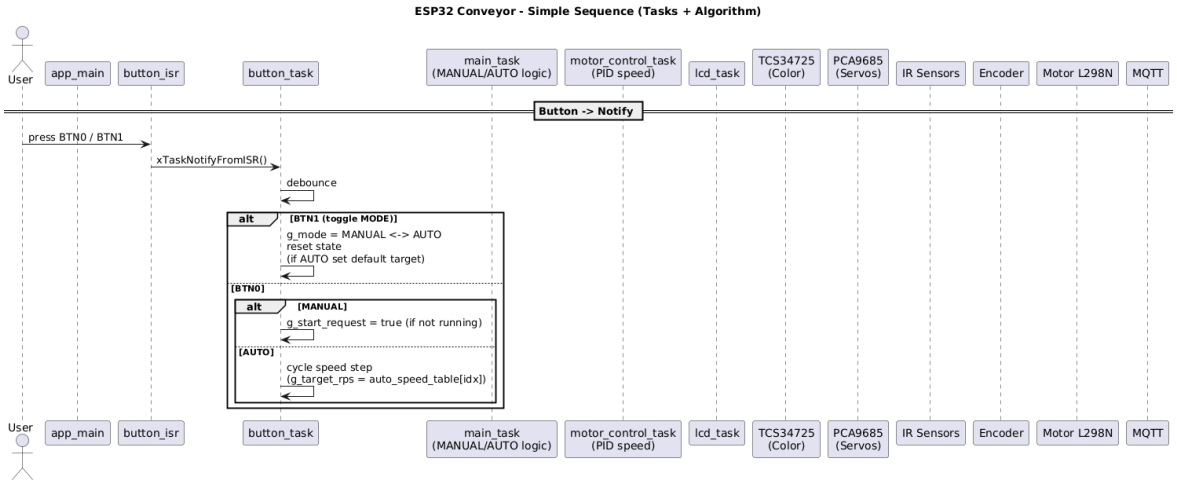
Đi vào Sequence Diagram của hệ thống, ta sẽ chia chúng ra làm 5 task chính:

- Init Task: đây là task được gọi đầu tiên khi bắt đầu khởi chạy hệ thống (hay boot ESP) được đặt ở đầu hàm main(), nhiệm vụ chính của nó sẽ là khởi tạo I2C, GPIO, Timer hay các thành phần khác nhỏ hơn bên trong các struct của components ( như LCD, sensors, motor, . . . chủ yếu là handlers). Và nó cũng đảm nhiệm tạo ra các task còn lại và gán handler cho task (nếu có).



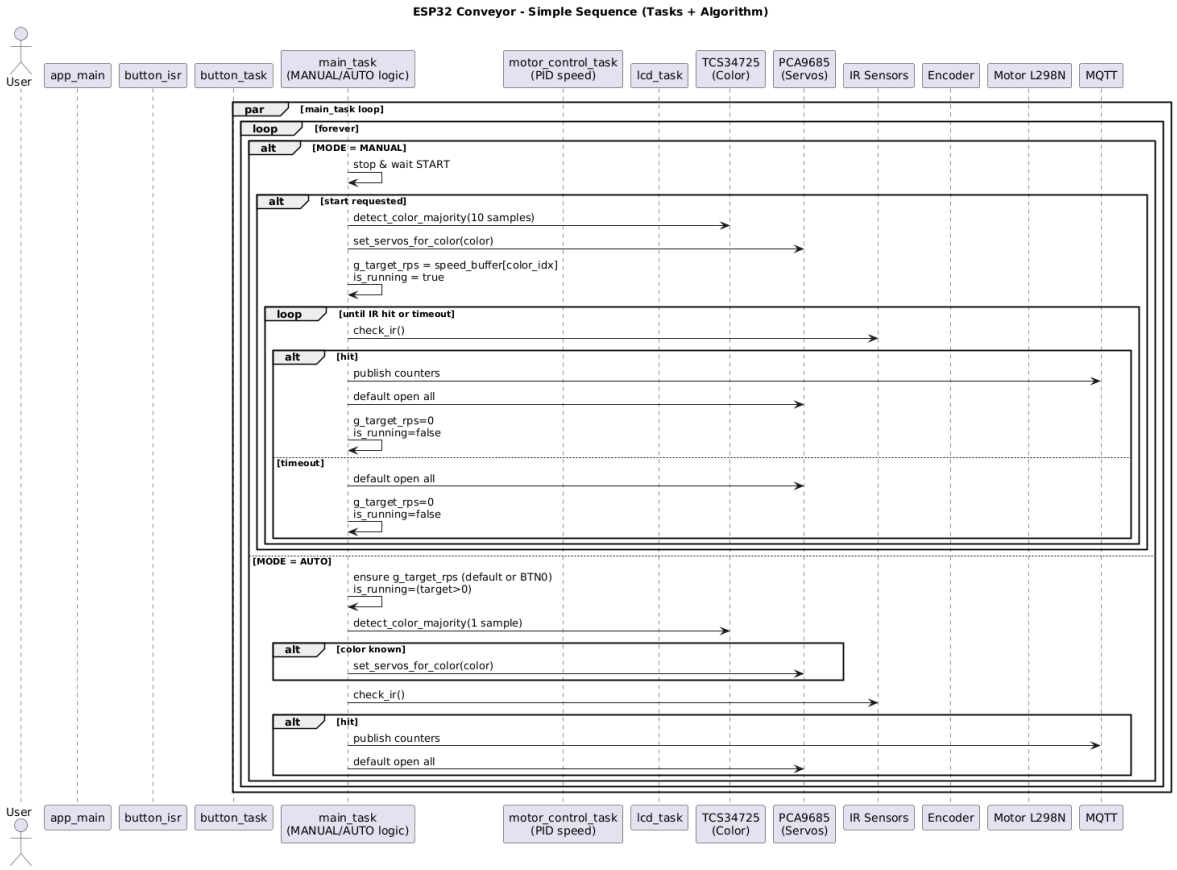

Hình 3.16: Init Task

- Button task: task này sẽ quản lý hai nút nhấn dược sử dụng trong hệ thống, được gọi khi ngắt xảy ra trên handler của task này. Khi được task hoạt động, nó sẽ kiểm tra xem nút vừa được bấm là nút nào, nếu là BTN1 thì sẽ toggle MODE (giữa MANUAL và AUTO) còn BTN0 thì sẽ thực hiện chức năng (START cho MANUAL và đổi tốc độ motor cho AUTO). Tất nhiên, task này không trực tiếp thực hiện các hành vi đó mà chỉ là thay đổi các biến toàn cục, các cờ và để cho các task khác lo. Vì BTN1 được gọi trước và continue nên khi xảy ra trường hợp cả 2 nút đều tích cực thì sẽ ưu tiên BTN1.



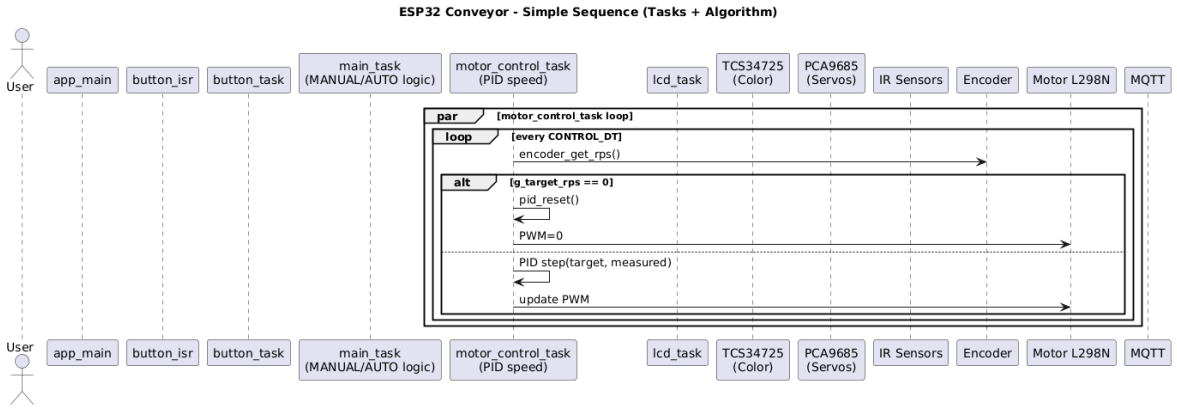

Hình 3.17: Button Task

- Main task (không phải hàm main, đây là task được tạo và được gọi trong main): Đây là task sẽ quản lý phần công việc chính của hệ thống. Task này sẽ kiểm tra biến g_mode (MODE được switch khi bấm nút) để chọn xem phải thực thi phần công việc của MODE nào. Đối với MANUAL, nó sẽ chờ cờ start của button task rồi mới thực hiện detect màu nhằm chọn ra màu chính của mẫu, đóng/mở servo, chạy băng tải, chờ cảm biến hồng ngoại khi vật rớt ra khỏi băng tải, thay đổi counter, publish dữ liệu lên MQTT và dừng băng tải quay lại vòng lặp cũ. Còn với AUTO, băng tải sẽ luôn chạy, phụ thuộc vào màu của vật mà đóng/mở servo tương ứng cho đến khi cảm biến hồng ngoại phát hiện vật, thay đổi counter và publish dữ liệu lên MQTT.



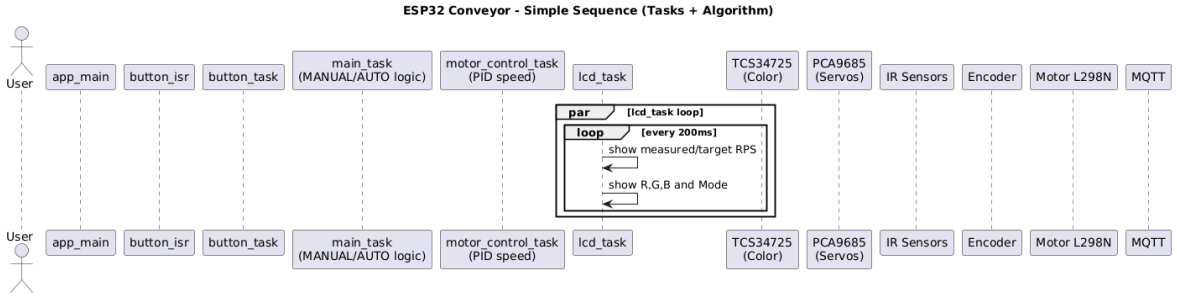

Hình 3.18: Main Task

- Motor task: task này sẽ đảm nhiệm điều chỉnh tốc độ của motor để sát với tốc độ setpoint nhất có thể. Tổng quan, nhiệm vụ của task này là: lấy tốc độ thực tế được đo bởi encoder, và thực hiện cập nhật PWM truyền vào cho L298N thông qua sai số giữa tốc độ thực tế và setpoint.




Hình 3.19: Motor Task

- LCD Task: thực hiện hiển thị dữ liệu lên LCD bao gồm: tốc độ thực tế trên tốc độ setpoint, RGB Counter (bộ đếm cho sản phẩm phân loại) và chế độ hiện hành của hệ thống.

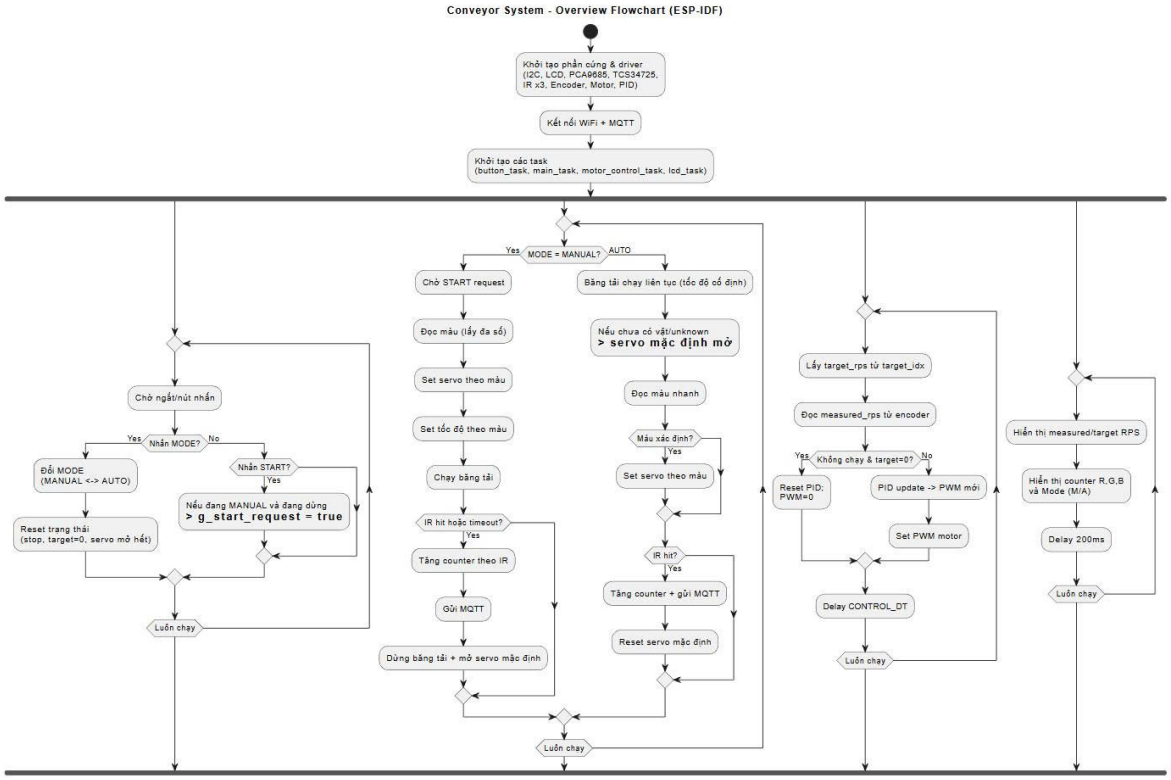



Hình 3.20: LCD Task

### 3.2.5 Thiết kế lưu đồ giải thuật

Lưu đồ giải thuật được sử dụng để mô tả trực quan toàn bộ luồng xử lý của hệ thống theo từng bước logic, từ giai đoạn khởi tạo đến quá trình vận hành liên tục. Thông qua các khối xử lý, khối điều kiện và mũi tên điều hướng, lưu đồ giúp thể hiện rõ trình tự thực hiện các tác vụ, các nhánh quyết định và mối quan hệ giữa các tiến trình song song trong hệ thống.

Đối với hệ thống băng tải phân loại màu, lưu đồ giải thuật đóng vai trò quan trọng trong việc làm rõ cách hệ thống phản ứng với các chế độ vận hành khác nhau (AUTO/MANUAL), cách xử lý sự kiện từ nút nhấn và cảm biến, cũng như cơ chế điều khiển động cơ và servo thông qua thuật toán PID. Lưu đồ là cơ sở để hiện thực chương trình trên nền tảng ESP-IDF và hỗ trợ kiểm tra, hiệu chỉnh hệ thống trong quá trình phát triển.




Hình 3.21: Tổng quan lưu đồ giải thuật.

#### 3.2.5.1 Các bước logic, giai đoạn và quá trình vận hành

**1. Giai đoạn khởi tạo hệ thống**

Lưu đồ bắt đầu với quá trình khởi tạo phần cứng và driver, bao gồm các khối giao tiếp (I2C, LCD, PCA9685), cảm biến màu TCS34725, cảm biến hồng ngoại, encoder, động cơ, servo và bộ điều khiển PID.

Sau đó, hệ thống tiến hành kết nối WiFi và MQTT, cho phép gửi dữ liệu vận hành và kết quả phân loại lên hệ thống giám sát. Khi kết nối hoàn tất, các task chính của hệ thống được khởi tạo như: button_task, main_task, motor_control_task và lcd_task. Đây là bước chuẩn bị cần thiết trước khi hệ thống đi vào trạng thái hoạt động.

**2. Phân nhánh chế độ hoạt động Manual/Auto**

Sau khi khởi tạo, lưu đồ chuyển sang khối quyết định MODE = MANUAL hay AUTO, từ đó hệ thống hoạt động theo hai luồng xử lý khác nhau.

**a. Chế độ Manual**

Ở chế độ MANUAL, hệ thống trước hết chờ yêu cầu START từ người dùng. Người dùng đặt sản phẩm cố định dưới cảm biến màu và nhấn nút START. Khi nhận được yêu cầu, hệ thống tiến hành:

+ Đọc dữ liệu màu (lấy giá trị trung bình để tăng độ ổn định).
+ Thiết lập vị trí servo tương ứng với màu đã xác định.
+ Thiết lập tốc độ băng tải theo màu.
+ Cho băng tải chạy.

Trong quá trình vận hành, hệ thống liên tục kiểm tra tín hiệu từ cảm biến hồng ngoại hoặc điều kiện timeout. Khi phát hiện sản phẩm đã đi qua:

+ Tăng bộ đếm sản phẩm.

+ Gửi dữ liệu kết quả phân loại lên MQTT.
+ Dừng băng tải và đưa servo về vị trí mặc định.

Sau đó, hệ thống quay trở lại trạng thái chờ cho chu kỳ MANUAL tiếp theo.

**b. Chế độ Auto**

Ở chế độ AUTO, băng tải chạy liên tục với tốc độ cố định ở mức thấp. Khi chưa phát hiện vật thể hoặc màu chưa xác định, servo được giữ ở vị trí mặc định mở. Hệ thống thực hiện đọc màu nhanh khi sản phẩm đi qua cảm biến. Khi màu được xác định:

+ Servo được điều khiển theo màu tương ứng để phân loại.
+ Hệ thống tiếp tục theo dõi tín hiệu từ cảm biến hồng ngoại.
+ Khi IR phát hiện sản phẩm đi qua, bộ đếm được tăng và dữ liệu được gửi lên MQTT.
+ Servo sau đó được reset về vị trí mặc định để sẵn sàng cho sản phẩm tiếp theo.

Chu trình này được lặp lại liên tục trong suốt thời gian hệ thống ở chế độ AUTO.

### 3. Xử lý nút và chuyển đổi chế độ

Song song với luồng xử lý chính, hệ thống luôn duy trì một nhánh xử lý nút nhấn. Khi người dùng nhấn nút MODE, hệ thống sẽ:

+ Chuyển đổi giữa chế độ MANUAL và AUTO.
+ Reset trạng thái hệ thống (dừng băng tải, đặt lại target, mở servo mặc định)

Nếu nút START được nhấn trong chế độ MANUAL khi băng tải đang dừng, hệ thống sẽ thiết lập cờ g_start_request để bắt đầu chu trình phân loại.

**4. Điều khiển tốc độ động cơ bằng PID**

Một nhánh xử lý riêng biệt trong lưu đồ thể hiện vòng điều khiển PID cho động cơ DC. Hệ thống liên tục:

+ Lấy giá trị tốc độ mục tiêu (target_rps).

+ Đọc tốc độ thực tế từ encoder (measured_rps).

Nếu tốc độ mục tiêu bằng 0 và băng tải không cần chạy, PID được reset và PWM được đặt về 0. Ngược lại, thuật toán PID sẽ tính toán giá trị PWM mới và cập nhật cho động cơ. Chu trình này được thực hiện theo chu kỳ điều khiển xác định (CONTROL_DT) và luôn chạy song song với các luồng khác.

**5. Hiển thị và giám sát hệ thống**

Cuối cùng, lưu đồ thể hiện một nhánh hiển thị LCD luôn hoạt động. Nhánh này định kỳ:

+ Hiển thị tốc độ thực tế và tốc độ mục tiêu của động cơ.

+ Hiển thị số lượng sản phẩm theo từng màu (R, G, B).

+ Hiển thị chế độ hoạt động hiện tại (AUTO/MANUAL).

Việc cập nhật được thực hiện theo chu kỳ cố định (ví dụ 200 ms) nhằm đảm bảo thông tin hiển thị rõ ràng và không ảnh hưởng đến hiệu năng hệ thống.

# Chương 4

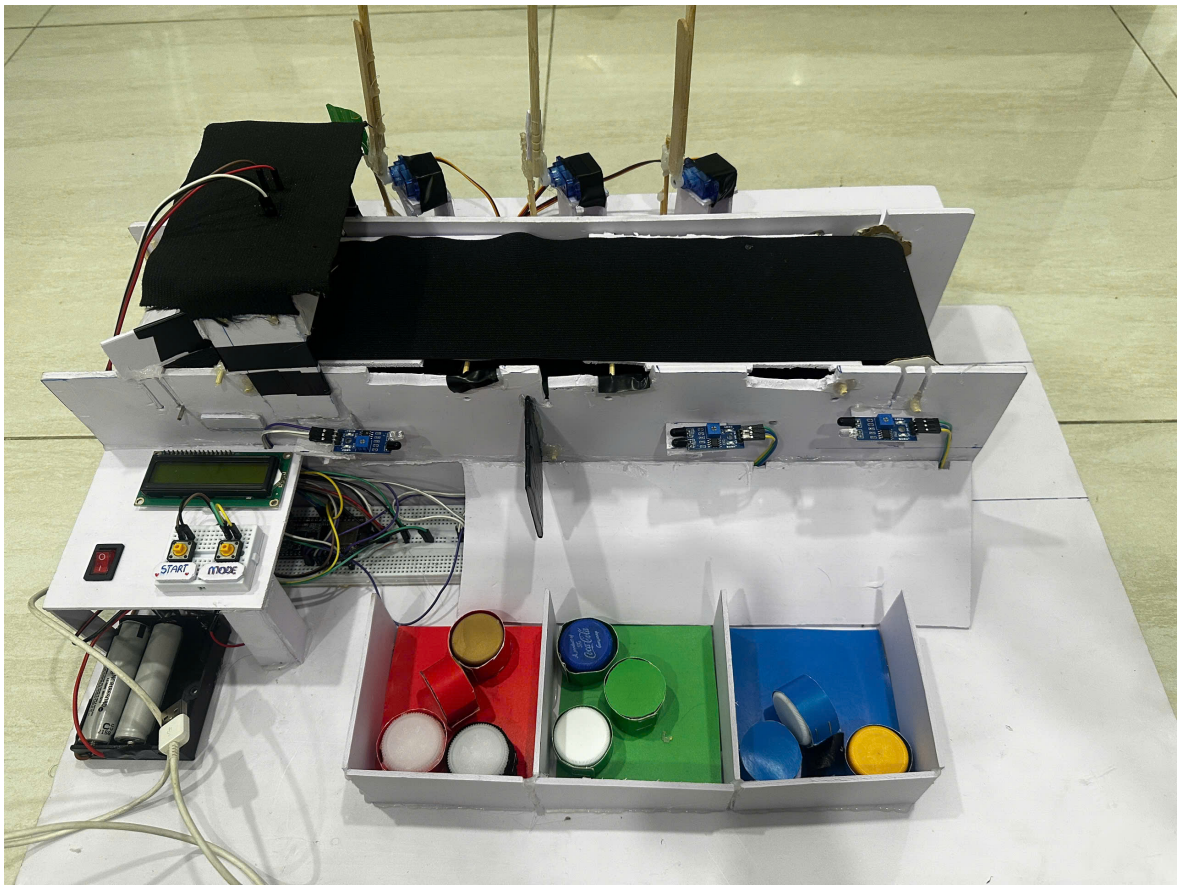
# Kết quả và đánh giá

## 4.1 Kết quả

### 4.1.1 Sản phẩm băng chuyền

Sau quá trình nghiên cứu, thiết kế và gia công, mô hình băng chuyền đã được hoàn thiện và vận hành theo đúng nguyên lý đề ra. Kết cấu tổng thể của băng chuyền đảm bảo khả năng hoạt động ổn định trong điều kiện thử nghiệm, đáp ứng yêu cầu cơ bản của hệ thống phân loại. Băng tải chuyển động liên tục, bề mặt băng tương đối phẳng nhờ đã được căn chỉnh phù hợp, góp phần hỗ trợ quá trình vận chuyển và phân loại vật thể.

Các cảm biến, vi xử lý và hệ thống dây nối được bố trí gọn gàng, rõ ràng và hợp lý, giúp hệ thống hoạt động ổn định cũng như thuận tiện cho việc quan sát và theo dõi trong quá trình vận hành. Mô hình băng chuyền sau khi hoàn thiện đã đáp ứng đầy đủ các yêu cầu đặt ra trong phạm vi đồ án môn học, làm cơ sở cho việc thực hiện các chức năng kiểm thử và đánh giá ở các phần tiếp theo.




Hình 4.1: Hình ảnh đóng gói băng chuyền hoàn thiện

### 4.1.2 Kết quả kiểm thử các chức năng

#### 4.1.2.1 Nhận diện màu sắc

Hệ thống sử dụng cảm biến màu TCS34725 kết hợp với các thuật toán nhận diện màu để xác định màu sắc của vật thể. Quá trình kiểm thử được thực hiện trong chế độ Manual với các mẫu màu RGB hợp lệ và mẫu không xác định (màu đen).

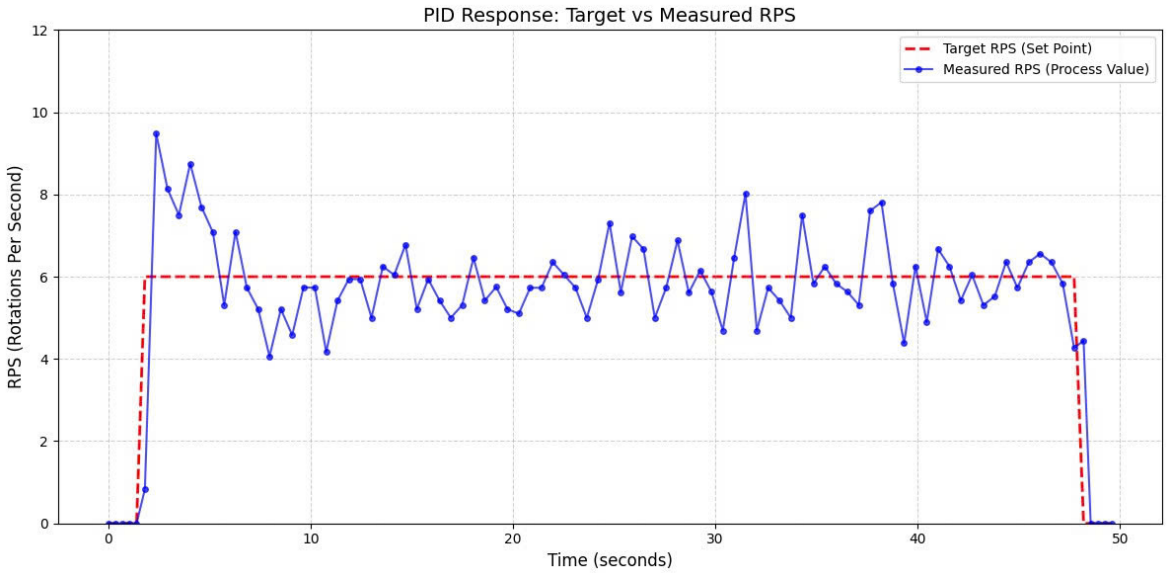
Kết quả cho thấy hệ thống nhận diện chính xác màu của các mẫu RGB, hiển thị đúng kết quả trên LCD và gửi dữ liệu thành công đến ứng dụng MQTT trên điện thoại. Đối với mẫu không xác định, hệ thống phân loại đúng là “other” và không hiển thị kết quả sai. Chức năng nhận diện màu hoạt động ổn định, không phát sinh lỗi trong quá

trình thử nghiệm.

#### 4.1.2.2 Bộ điều khiển băng tải PID

Chức năng điều khiển tốc độ băng tải được kiểm thử thông qua bộ điều khiển PID kết hợp với encoder đo tốc độ. Các thử nghiệm được thực hiện với nhiều giá trị tốc độ mục tiêu khác nhau, tương ứng với từng màu sản phẩm.

Kết quả cho thấy tốc độ đo được hội tụ về giá trị đặt và duy trì ổn định trong giới hạn sai số cho phép ($\pm 2RPS$). Đáp ứng của bộ điều khiển PID mượt, không xảy ra dao động lớn hay mất ổn định. Trong quá trình đo thực nghiệm, tồn tại sai lệch nhỏ giữa tốc độ trục và tốc độ băng tải với mức chênh lệch khoảng $\pm 1-2\%$, tuy nhiên hệ thống vẫn đảm bảo vận hành ổn định. Dữ liệu log được sử dụng để vẽ đồ thị tốc độ theo thời gian nhằm xác nhận tính chính xác của bộ điều khiển.




Hình 4.2: Response Graph PID

Hình minh họa thể hiện đáp ứng tốc độ của hệ thống theo thời gian, trong đó đường biểu diễn tốc độ mục tiêu và tốc độ đo được cho thấy sự hội tụ nhanh và dao

động nhỏ quanh điểm làm việc. Điều này khẳng định bộ điều khiển PID đáp ứng tốt yêu cầu điều khiển tốc độ băng tải và đảm bảo hệ thống vận hành ổn định.

#### 4.1.2.3 Phân loại bằng servo

Servo được điều khiển thông qua module PCA9685 để thực hiện thao tác mở cửa phân loại theo từng màu. Kết quả thử nghiệm cho thấy servo quay đúng góc tương ứng với màu được nhận diện, đảm bảo phần lớn vật thể được đưa vào đúng ngăn phân loại. Cơ cấu cơ khí hoạt động ổn định, không xuất hiện hiện tượng kẹt hoặc sai lệch vị trí trong quá trình vận hành.

Trong quá trình thử nghiệm thực tế, ghi nhận tồn tại xác suất nhỏ (khoảng 2–5%) vật thể không rơi chính xác vào ngăn phân loại do cánh tay gạt servo có độ cứng chưa cao. Tuy nhiên, hiện tượng này không ảnh hưởng đáng kể đến hoạt động tổng thể của hệ thống trong phạm vi đồ án.

#### 4.1.2.4 Nội dung hiển thị LCD

Màn hình LCD được kiểm tra về nội dung hiển thị tốc độ băng tải, bộ đếm sản phẩm theo màu và trạng thái hệ thống. Kết quả cho thấy LCD cập nhật thông tin định kỳ, hiển thị đúng giá trị tốc độ thực tế và tốc độ mục tiêu.

Các bộ đếm R, G, B tăng chính xác theo số lượng sản phẩm đã phân loại. Ngoài ra, LCD còn hiển thị đúng chế độ hoạt động hiện tại của hệ thống (Manual hoặc Auto), giúp người dùng dễ dàng theo dõi trạng thái.

#### 4.1.2.5 Nút điều khiển

Các nút nhấn được kiểm thử nhằm đánh giá khả năng chuyển đổi chế độ và kích hoạt chu trình hoạt động của hệ thống. Kết quả cho thấy nút chuyển chế độ cho phép hệ thống chuyển đổi an toàn giữa chế độ Manual và Auto, đồng thời servo được đưa về trạng thái an toàn khi thay đổi chế độ.

Nút Start chỉ có hiệu lực trong chế độ Manual và kích hoạt đúng một chu trình

hoạt động. Trong quá trình thử nghiệm, không ghi nhận hiện tượng kích hoạt nhầm, bỏ sót sự kiện hoặc nhiễu tín hiệu do nhấn liên tiếp hay nhấn giữ lâu.

#### 4.1.2.6 Bộ đếm sản phẩm (IR sensor)

Chức năng đếm sản phẩm được kiểm thử bằng cảm biến hồng ngoại IR. Kết quả cho thấy mỗi vật thể đi qua chỉ được ghi nhận một lần, không xảy ra hiện tượng đếm lặp cho cùng một vật trong quá trình vận hành.

Cơ chế chờ nhả tín hiệu của cảm biến IR hoạt động hiệu quả, đảm bảo độ chính xác của bộ đếm trong nhiều lần thử nghiệm liên tiếp. Trong quá trình kiểm thử thực tế, hệ thống ghi nhận hiệu suất nhận diện vật thể rơi xuống ô phân loại ở mức cao, tuy nhiên vẫn xuất hiện một số trường hợp không phát hiện được vật thể có màu đen.

## 4.2 Đánh giá

Bên cạnh các kết quả đạt được, hệ thống vẫn tồn tại một số hạn chế chủ yếu liên quan đến kết cấu cơ khí của băng chuyền. Do băng chuyền được chế tạo từ vật liệu bìa Formex, mặc dù có độ cứng nhất định, trong quá trình vận hành vẫn xuất hiện rung lắc nhẹ, từ đó có thể gây sai số trong quá trình đo tốc độ và phân loại sản phẩm. Hiện tượng rung lắc này ảnh hưởng gián tiếp đến độ ổn định của chuyển động băng tải cũng như độ chính xác của các cảm biến.

Ngoài ra, các khớp nối cơ khí trong hệ thống chủ yếu được cố định bằng keo nến và keo 502 nên độ bền chưa cao, đặc biệt khi hệ thống hoạt động trong thời gian dài. Điều này có thể dẫn đến hiện tượng sứt, bong hoặc lệch khớp nối, làm thay đổi vị trí tương đối giữa các bộ phận cơ khí và gây sai số trong quá trình vận hành. Khớp nối giữa động cơ và trục quay được cố định bằng keo cũng làm giảm độ ổn định của chuyển động, nhất là khi động cơ hoạt động ở tốc độ cao.

Bên cạnh đó, độ bám giữa dây thun băng tải và các trục quay chưa hoàn toàn tối ưu, có thể xảy ra hiện tượng trượt trong một số thời điểm, từ đó gây ra sai lệch giữa tốc độ trục động cơ và tốc độ thực tế của băng tải. Những hạn chế này là nguyên nhân

chính dẫn đến các sai số nhỏ đã được ghi nhận trong quá trình kiểm thử.

Tổng thể, các hạn chế trên xuất phát từ điều kiện vật liệu và phương pháp gia công trong phạm vi đồ án môn học. Mặc dù còn tồn tại sai số, hệ thống vẫn đảm bảo hoạt động ổn định và đáp ứng được các yêu cầu cơ bản đặt ra.

# Chương 5

# Kết luận và hướng phát triển

## 5.1 Hướng phát triển

### 5.1.1 Phát triển về cơ khí

Trong các nghiên cứu tiếp theo, hệ thống băng chuyền có thể được cải tiến bằng cách nâng cấp vật liệu chế tạo nhằm tăng độ cứng và độ bền của kết cấu. Việc thay thế vật liệu bìa Formex bằng các vật liệu có độ ổn định cao hơn như mica, nhôm hoặc thép mỏng sẽ giúp giảm rung lắc và hạn chế sai số trong quá trình vận hành. Bên cạnh đó, các khớp nối cơ khí nên được cố định bằng vít hoặc khớp cơ khí chuyên dụng thay cho keo dán, nhằm nâng cao độ bền và độ ổn định khi hệ thống hoạt động trong thời gian dài.

Ngoài ra, cơ cấu truyền động cần được tối ưu để tăng độ bám giữa băng tải và trục quay, hạn chế hiện tượng trượt gây sai lệch tốc độ. Cánh tay gạt servo cũng có thể được cải tiến bằng cách sử dụng vật liệu cứng hơn và tối ưu hình dạng cơ khí, từ đó giảm xác suất vật thể rơi lệch khỏi ngăn phân loại.

### 5.1.2 Phát triển về bộ điều khiển tốc độ

Về mặt điều khiển, bộ điều khiển PID hiện tại có thể được tối ưu thêm thông qua việc tinh chỉnh tham số nhằm giảm sai số và cải thiện thời gian đáp ứng. Trong các nghiên cứu nâng cao, có thể xem xét áp dụng các phương pháp điều khiển tiên tiến hơn như PID thích nghi, điều khiển mờ hoặc điều khiển kết hợp nhằm tăng khả năng thích ứng của hệ thống trong các điều kiện tải và môi trường khác nhau.

Bên cạnh đó, việc bổ sung thêm cảm biến đo trực tiếp tốc độ băng tải thay vì chỉ đo tốc độ trục động cơ sẽ giúp tăng độ chính xác của tín hiệu phản hồi, từ đó nâng cao hiệu quả điều khiển và giảm sai số giữa tốc độ đặt và tốc độ thực tế.

### 5.1.3 Phát triển về cảm biến

Đối với các cảm biến, việc nhận diện có thể nâng cao bằng cách cải tiến thuật toán xử lý tín hiệu hoặc kết hợp nhiều loại cảm biến khác nhau. Đặc biệt, đối với cảm biến IR, có thể bổ sung các cơ chế lọc nhiễu hoặc sử dụng các loại cảm biến có khả năng phát hiện vật thể tốt hơn trong trường hợp vật có màu tối hoặc khả năng phản xạ kém.

Ngoài ra, việc cải thiện điều kiện chiếu sáng cho cảm biến màu, kết hợp với các thuật toán hiệu chỉnh và bù sai số môi trường, sẽ giúp nâng cao độ ổn định và độ chính xác của quá trình nhận diện màu sắc trong các điều kiện vận hành khác nhau.

## 5.2 Kết luận

Trong phạm vi đồ án môn học, nhóm đã thực hiện thiết kế và xây dựng thành công hệ thống băng chuyền phân loại màu sắc sử dụng vi xử lý ESP32. Hệ thống được tích hợp đầy đủ các thành phần như cảm biến màu, động cơ DC, servo, cảm biến hồng ngoại và bộ điều khiển PID nhằm phục vụ cho quá trình phân loại sản phẩm.

Qua quá trình thử nghiệm, hệ thống cho thấy khả năng hoạt động tương đối ổn định. Chức năng nhận diện màu sắc hoạt động chính xác với các mẫu màu cơ bản, băng chuyền được điều khiển tốc độ hiệu quả nhờ bộ điều khiển PID, giúp tốc độ

động cơ bám sát giá trị đặt. Cơ cấu phân loại bằng servo, màn hình hiển thị LCD và các nút điều khiển đều hoạt động đúng chức năng, hỗ trợ tốt cho việc vận hành và theo dõi trạng thái hệ thống.

Bên cạnh những kết quả đạt được, hệ thống vẫn còn một số hạn chế, chủ yếu liên quan đến phần cơ khí và đặc tính của cảm biến, do điều kiện vật liệu và phạm vi thực hiện đồ án. Tuy nhiên, các sai số này ở mức chấp nhận được và không ảnh hưởng nhiều đến hoạt động tổng thể của hệ thống. Những hạn chế này cũng là cơ sở để nhóm đề xuất các hướng cải tiến và phát triển trong thời gian tới.

Nhìn chung, đồ án đã giúp nhóm vận dụng được các kiến thức đã học về hệ thống nhúng, điều khiển PID và cảm biến vào một mô hình thực tế. Kết quả đạt được đáp ứng yêu cầu của môn học và tạo nền tảng để tiếp tục nghiên cứu, cải tiến hệ thống trong các đề tài sau.

# Phụ lục

| |
|---|
| Link file excel Template_plan:<br>https://docs.google.com/spreadsheets/d/1VuJUVkotevS37AYuso59GtwrOtOUxq34oSXoekWLI2w/edit?usp=sharing |
| Link video demo (Bản chức năng trước khi chỉnh sửa):<br>https://drive.google.com/drive/folders/1VbBhvso0Ip7ruKOpfUnHTzwYDVnQc1GH?usp=sharing |
| Link video demo (Bản sau khi chỉnh sửa thêm vào chức năng tăng/giảm tốc độ của băng chuyền):<br>https://drive.google.com/drive/folders/1EpNU9o3i_j1hvZeOCtnL1cw4p5Zp4SV8?usp=sharing |
| Link github:<br>https://github.com/quocbao8925/EmbSysProject-UIT-CE224.Q11.git |

# TÀI LIỆU THAM KHẢO

## Tài liệu tiếng Việt


[1] HShop.vn. Cảm biến màu sắc TCS34725 giao tiếp I2C và UART, urlseen 17 december 2025. url: https://hshop.vn/cam-bien-mau-sac-tcs34725-giaotiep-i2c-va-uart

[2] HShop.vn. Cảm biến vật cản hồng ngoại IR Infrared Obstacle Avoidance, urlseen 18 december 2025. url: https://hshop.vn/cam-bien-vat-can-hong-ngoai-v1-2

[3] HShop.vn. Đĩa Encoder 20 xung Động Cơ DC Giảm Tốc V1, urlseen 18 december 2025. url: https://hshop.vn/dia-encoder-20-xung-dong-co-dc-giam-toc-v1

[4] HShop.vn. Động cơ DC giảm tốc V1 Dual Shaft Plastic Geared TT Motor, urlseen 17 december 2025. url: https://hshop.vn/dong-co-dc-giam-toc-v1-dual-shaft-plastic-geared-tt-motor

[5] HShop.vn. Động cơ RC Servo 9G 0-180°, urlseen 18 december 2025. url: https://hshop.vn/dong-co-rc-servo-9g

[6] HShop.vn. Mạch điều khiển động cơ L298 DC Motor Driver, urlseen 18 december 2025. url: https://hshop.vn/mach-dieu-khien-dong-co-dc-l298

[7] HShop.vn. Module Encoder 20 xung, urlseen 18 december 2025. url: https://hshop.vn/module-encoder-20-xung

# Tài liệu tiếng Anh

[8] Lewis, H. R. Finite Automata, urlseen 19 december 2025. url: https://scholar.harvard.edu/files/harrylewis/files/fa_0.pdf

[9] Ufot, O. N., Ekpoudom, I. I. and Akpan, E. A., Development of a Fuzzy Logic Controller for Industrial Conveyor, *American Journal of Science, Engineering and Technology*, jourvol 2, number 3, pages 77–82, 2017.

[10] Umorn, M. A., Essien, A. O. and Ekpoudom, I. I., Design and Implementation of Conveyor Line Speed Synchroniser for Industrial Control Applications: A Case Study of Champion's Breweries PLC UYO, *Nigerian Journal of Technology*, jourvol 35, number 3, pages 618–626, 2016.